

Designing a Programming Language for Video Game Map Creation

cs-25-dat-8-05

8th Semester Project





Title:

Designing a Programming Language for
Video Game Map Creation

Theme:

Design, Definition, and Implementation of
Programming Languages

Project Period:

Spring Semester 2025

Project Group:

CS-It8, Group 5
cs-25-dat-8-05

Participant(s):

Eske Klintø Larsen
Mathias Holm Mariager
Thormod Jakob Schøn Merstrand
Toms Vanders
Ashok Dahit Tharu
Kareena Khadka
Faustas Anulis

Supervisor:

Frederik M. Bønneland

Page Numbers:

76

Date of Completion:

May 27, 2025

Abstract:

This project covers the domain of video game creation and aims to create a programming language for map generation. First, we analyze different aspects of the game development industry, in specific the relation between triple-A and small-scale developers, as well as current tools for video game development, and existing ways developers do map generation. Based on the analysis, we defined the specifications for a domain-specific language (DSL) that consists of three core instructions: Map, Room, and Connect. Afterwards, the project describes our implementation of the handwritten compiler, which translates source code to machine code, following the classic compiler pipeline: lexical, syntax, semantic analysis, intermediate code, and machine code generation. The semantic analyzer guarantees that rooms and connections are unique and constraints are validated during compile time. The compilers final output is a graph structure in NASM assembly, built around the naming convention of the three core instructions and a C interface to utilize it.

Contents

1	Introduction	1
2	Problem Analysis	2
2.1	Tools for Video Game Creation	2
2.2	Indie Games vs Triple-A	2
2.3	Map Generation in Video Games	3
2.4	Problem Statement	5
3	Theory	6
3.1	Aspects of Programming Languages	6
3.1.1	DSL and GPL	6
3.1.2	Static Typing vs Dynamic Typing	6
3.1.3	Levels of Programming Languages	7
3.1.4	Programming Paradigms	7
3.2	Compiler Theory	8
3.2.1	Lexical Analysis	9
3.2.2	Syntax Analysis	10
3.2.3	Type Checking	10
3.2.4	Intermediate Code Generation	11
3.2.5	Register Allocation Generation	11
3.2.6	Machine Code Generation	12
3.3	Alternative Execution Models	12
3.3.1	Interpreter	12
3.3.2	Transpiler	13

4	Design of The Language	14
4.1	Language Design Decisions	14
4.1.1	High Level vs. Low Level Language	14
4.1.2	Programming Paradigms	14
4.1.3	Static vs Dynamic Typing	15
4.1.4	DSL vs. GPL	15
4.1.5	Compiler vs. Interpreter vs. Transpiler	15
4.2	Syntax and Semantics Design	16
4.2.1	Pseudo Code	16
4.2.2	Basic Syntax Rules	16
4.2.3	The Abstract Syntax	17
4.2.4	The Concrete Syntax	18
4.2.5	Inference Rules for Static Semantics	19
5	Implementation	21
5.1	Common	23
5.1.1	Compiler State	23
5.1.2	Definitions	24
5.1.3	Error Handler	25
5.2	Lexical Analyzer	26
5.3	Abstract Syntax Tree Implementation	32
5.4	Parser	35
5.4.1	Parsing the Pseudo Code	41
5.5	Semantic Analysis	43
5.5.1	Semantic Analysis Implementation	43
5.5.2	Symbol Table	43
5.5.3	Semantic Analyzer	46
5.6	Intermediate Language	48
5.7	Machine Code	52
5.7.1	Code Generation	52

5.7.1.1	Generate Map	52
5.7.1.2	Generate Config	55
5.7.1.3	Generate Interfaces	57
5.7.2	Handwritten	59
5.7.2.1	Core	60
5.7.2.2	Convert	62
5.7.2.3	Stack	64
5.7.2.4	DFS	66
5.7.2.5	Graph_Walk	67
6	Discussion	70
6.1	Appealing to Target Group	70
6.2	Solving the Problem Statement	71
6.3	Features	71
6.4	Libraries	72
6.5	Test Integration	72
7	Future Work	73
7.1	Acceptance Testing	73
7.2	Stress Testing	73
7.3	Additional Operators	74
7.4	Interpreter	74
7.5	Custom Room Types	74
7.6	Semi-Auto and Auto Map Generation	74
7.7	Asynchronous Map Generation	75
8	Conclusion	76

Chapter 1

Introduction

The creation of video games has become a huge industry, with the video game industry being worth more than triple the movie and music industry combined [16]. The creation of a single video game is not a simple process, with even simple games needing a good soundtrack, game mechanics, UI, and a world for the player to utilize and explore.

This project aims to cover the map creation aspect of video game development. We want to provide our target group of mainly small-scale developers, indie game creators, and hobbyists, with a powerful and accessible tool to quickly prototype and generate map environments. To support our target group, the language is designed with simplicity in mind, requiring only basic programming knowledge, such as familiarity with data types, statements, variables, and syntax. The language aims to streamline the creation of map structures, such as mazes, dungeons, or labyrinths, in game development. With this language, developers will be able to define different map components in a simple way. Using a graph-based approach, it allows users to define components such as rooms and connections. It includes the possibility to define different map constraints, namely, the max connections per room and a max threshold of rooms.

This project will first cover existing tools for video game creation, as well as how some games do their map generation. Next, we cover the theory behind programming languages and what specifies a compiler. This is followed by a description of our design choices made for the language, leading up to the next chapter, where we will explain the implementation of the language. Finally, there is a reflection upon the project where we discuss different aspects of the language, what future development is needed for the language, and lastly, a conclusion on the project as a whole.

Chapter 2

Problem Analysis

This chapter includes an analysis of the problem domain within game development. We look at some of the existing tools for video game creation, as well as the differences between triple-A game studios and indie game developers. Finally, we will go through how some indie games do their map generation.

2.1 Tools for Video Game Creation

The development of video games has come a long way. What started with games written in the Assembly language and hardware specifically tailored to run the game [12] has turned into an industry where everyone with a basic understanding of programming languages can start their video game-making journey. A plethora of tools exist to aid developers, and among some of the more popular ones are Unity [25] and Unreal Engine [7]. Both Unity and Unreal Engine provide their users with a game engine as well as a fully-fledged development environment to help users tweak everything related to their engines. With these development tools publicly available, it has allowed a new actor to enter the video game market.

2.2 Indie Games vs Triple-A

Video game development has throughout its lifetime turned into a field with two actors. On one side, you have the big game companies, the so-called "triple-A" studios, that are financially robust and have access to a great workforce [11]. On the other side, you have the rising competitor of indie game developers, which, due to their small team of developers, tend to focus on creating unique gaming experiences [11].

The rise of indie games can, for example, be seen on one of the biggest online game distribution services, Steam [26]. Of the 13,000 games released on Steam's platform in 2024, 99% of these were indie games, with indie games also making up 58% of the 504 million game units sold on Steam in 2024 [27]. But for the indie game developers to end up with a game to add to this statistic, they have to start somewhere.

While Unity and Unreal Engine provide their users with all the tools needed to create a video game, newcomers or indie game developers might not want or need a full-fledged solution to create what they deem necessary for their game. Since there is a good chance these developers would still see game development as a hobby, they might want to be able to quickly create an environment to test potential ideas for their game.

2.3 Map Generation in Video Games

Creating a map remains a key aspect of video game creation. While the triple-A companies have the capacity to create beautiful and expansive 3D maps, indie developers might tend to go with smaller-scale maps. One way indie game developers create their maps is by using a room-based approach where rooms are "glued" together to create the full map. Examples of this are the indie rogue-likes "The Binding of Isaac: Rebirth" and "Enter the Gungeon" which have two different approaches to turning their rooms into a full map.

In "The Binding of Isaac: Rebirth", the rooms all follow a 1x1, 1x2, or 2x2 square shape. Every room can have 1-8 doors, depending on their size, and these doors act as the "glue" where rooms are glued together using their door placement [4]. An example of a full map can be seen in Figure 2.1.



Figure 2.1: Minimap of The Binding of Isaac: Rebirth [4]

"Enter the Gungeon" differs from "The Binding of Isaac: Rebirth" since it allows rooms to have all kinds of shapes. This means that it has to have another approach to "gluing" rooms together since their shapes make it impossible for them to fit together directly. Therefore, they utilize hallways as connectors between rooms that can have any shape that may be needed to connect rooms [5]. An example of a full map for "Enter the Gungeon" can be seen in Figure 2.2.



Figure 2.2: Minimap of Enter The Gungeon [5]

"The Binding of Isaac: Rebirth" and "Enter the Gungeon" are just two ways of doing the room-based map generation, and there remains a multitude of other ways to create a map for a video game. Therefore, a tool specifically tailored to create video game maps might be needed to aid developers in creating the perfect map for their game.

2.4 Problem Statement

Based on the previous sections, the following problem statement was developed:

How can we design a programming language specifically tailored to generating maps for small-scale games?

Chapter 3

Theory

In this chapter, we will go through the relevant theory. We will start by explaining a more general theory about programming languages. Afterwards, we go into specific theory surrounding compilers. Finally, we will also cover alternative methods to compilers, specifically interpreters and transpilers.

3.1 Aspects of Programming Languages

3.1.1 DSL and GPL

A domain-specific language (DSL) provides a notation tailored towards an application domain and is based on the relevant concepts and features of that domain.^[6] It employs specialized syntax and abstractions according to the requirements of the domain and enables clarity, efficiency, and precision. Unlike a general-purpose language, a DSL is narrow in scope, mainly focusing on the problems and tasks relevant to its targeted area. DSL offers significant advantages in terms of productivity, reliability, optimizability, and lower maintenance costs. ^[13].

A general-purpose language (GPL) is a flexible language that can be used in multiple domains. These languages often include procedural, functional, and object-oriented programming languages. It provides the capability to create new data types and define operations on them directly within the language itself. Compared to DSL, it's designed to be general, meaning fewer boundaries are applied to the language.^[13]

3.1.2 Static Typing vs Dynamic Typing

Static typing is a method where type checks are performed before execution, ensuring that type errors are caught early and allowing for compiler optimizations. Statically typed languages enable programmers to catch type errors during compilation, allowing them to address issues immediately. This prevents errors from surfacing at runtime, when the programmer may be focused on other tasks or when the software has already been deployed.

On the other hand, dynamic typing allows type checks to occur at runtime, rather than before execution. This allows for greater flexibility and faster development since programmers do not need to specify types explicitly. However, dynamic typing can lead to runtime errors that are harder to detect and debug. Static typing aids performance and maintainability in grand-scale projects, while dynamic typing offers convenience and usability in prototyping and scripting. [8]

3.1.3 Levels of Programming Languages

In the past, using computers was expensive, so everyone focused on writing programs carefully rather than making programming easier. It was discovered that making programs at the very low level was not only difficult but also highly vulnerable to human error. [19]

Due to the drawbacks of low-level languages, high-level languages appeared in the 1950s. [19] The programming languages are low-level and high-level, based on the level of abstraction from the hardware.

Low-level Languages

Low-level languages are intended to closely communicate with a computer's hardware. As an example, weakly typed low-level languages such as C lack type safety and allow direct access to memory, which can lead to memory misuse and errors. [19] Fine-grain control that low-level languages provide is suitable for creating applications where performance and hardware interactions are needed.

High-level Languages

High-level languages were invented to minimize difficulties when programming at the machine level. They use formalized notations that can express programs in a more readable and understandable way for humans. High-level concepts are translated into machine code, so it is not possible to express more ideas than what machine language allows. However, this helps developers write code faster and provides a structured syntax that can be verified against a formal grammar and type system to catch errors. [19]

3.1.4 Programming Paradigms

Programming paradigms are used to describe the fundamental approach to programming on a computer. The paradigms can refer to multiple aspects of programming, such as how the data flow is determined, the execution order, the structure of a program, or a combination. This means that there are multiple programming paradigms for a language to follow, however, programming languages will always support one or more programming paradigms.[19] Some of the main programming paradigms are:

Imperative Programming Imperative programming is fundamentally constructed around the idea of assigning values to parts of the running program's memory, and where execution of the code is structured as a sequence of different statements. This way of programming means that identical statements can produce different outcomes due to modification of global variables. This effect is called a side effect.[19]

Object-Oriented Programming (OOP) Object-oriented programming (OOP) is fundamentally constructed on the concept of having different objects. Each object is an instance of a class and can contain both data and a method for the class. [19]

Declarative Programming A declarative programming language will, as the name suggests, declare what the program should do rather than how the program should do it. This form of language relies on the implementation of the language to have an algorithm to solve a certain request in the background, without being shown directly in the written code. [19]

Functional Programming Functional programming is fundamentally constructed on the same idea of functions in mathematics, where a function f is given a value x and then returns a value $f(x)$. This means that data in a functional programming language is only communicated through function arguments and return values. A purely functional program has referential transparency, meaning that an identifier will have no assignment or side effect but always refer to the same value, which allows for mathematical reasoning when programming. [19]

Logical Programming Logical programming is fundamental based on the idea of setting up some rules or logical statements for the relation between data in a program, so instead of defining a step-by-step solution, the language is used to set up the logical rules for data. [19]

3.2 Compiler Theory

Designing and building computers using machine code is a complex and tedious task. Writing machine code is error-prone and difficult to manage. To simplify this process, machine code is often transformed into a collection of simple commands that make up high-level languages, balancing between the complexity of machine code and readability. This transformation is carried out by a compiler that breaks down the task into multiple stages for efficiency. Mogensen [19] divides this process into six phases (Figure 3.1), each with its specific responsibilities.

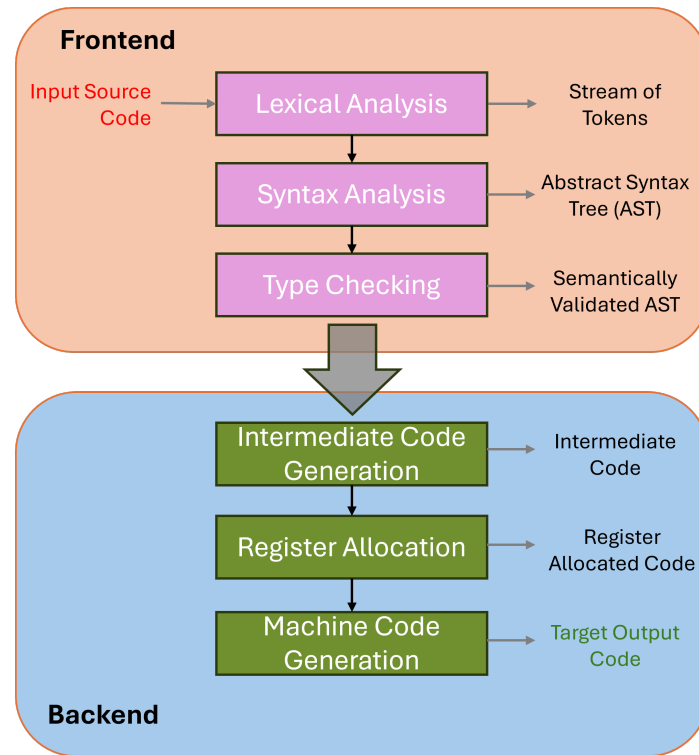


Figure 3.1: Compiler pipeline showing the phases from source code to machine code.

The compiler's frontend focuses on understanding program structure. It converts raw source text into an abstract syntax tree (AST) while verifying language rules, much like how a human proofreader checks grammar and syntax. The backend then transforms this AST into efficient machine code, handling architecture-specific optimizations and resource management. In the following subsections, we examine each phase's role in this transformation pipeline.

3.2.1 Lexical Analysis

The term "lexical" refers to "pertaining to words" [18]. In the context of programming languages, it refers to entities such as variable names, numbers, and keywords. These entities are known as tokens. [18]

A lexical analyzer, also known as a lexer or scanner, is responsible for converting the input source code into tokens. It processes a sequence of characters and applies a set of rules, often regular expressions, to classify and define each sequence. For example, a sequence like `int x = 10;` can be broken down into tokens: `int`, `x`, `=`, `10`, and `;`.

The primary role of the lexer is to divide the source code into chunks (tokens) and recognize patterns in the input to classify sequences as tokens. Additionally, filter non-essential elements such as whitespace, newlines, and comments while flagging illegal characters. These tokens provide a structured representation of the code, simplifying syntax analysis.

In theory, lexical analysis and syntax analysis can be combined into a single phase, where the tokenization process directly feeds into syntax analysis without the need for a separate lexer. Once the tokens are identified, they are delegated to the parser for further analysis.[18]

3.2.2 Syntax Analysis

Syntax analysis in compiler design ensures that source code adheres to grammatical rules. It processes tokens from lexical analysis and structures them into an abstract syntax tree (AST) for further processing. Based on Context-Free Grammar (CFG), parsing can be top-down (e.g., recursive descent, LL parsing) or bottom-up (e.g., LR parsing, SLR, LALR, CLR). Top-down parsing starts from the top (root), expanding rules, while bottom-up parsing constructs the tree from the bottom (leaves), efficiently handling complex grammars. Error handling mechanisms like panic mode recovery, phrase-level recovery, error productions, and global correction help manage syntax errors. The AST optimizes structure by omitting redundant details, ensuring efficiency. Syntax analysis is crucial for validating code correctness before further compilation stages. [1]

The choice of parsing technique has a big impact on compiler efficiency and complexity. For example, LL parsers are simpler to implement but struggle with left-recursive grammars, while LR parsers handle a broader range of languages but require more complex table generation. In modern compilers, tools like Yacc, Bison, and ANTLR automate parser generation, reducing manual effort and minimizing errors. A comparative study by Latif et al. [14] highlights that tools like ANTLR and JavaCC offer enhanced parsing features and greater support for complex grammar definitions compared to older tools like Yacc or Bison. Handwritten parsers remain a very valid choice as well. They are preferred in cases where handling complex error scenarios or when fine-grained control over parsing performance and memory management is essential. GCC [9] and Clang [15] compilers are popular examples that rely on handwritten parsers.

3.2.3 Type Checking

Type checking is often the last phase of the front-end. This phase verifies semantic correctness, ensuring that each operation is performed on arguments of appropriate types. It typically operates by traversing the AST produced during syntax analysis. This traversal uses attribute grammar techniques, where synthesized attributes propagate type information upwards from expressions to declarations, and inherited attributes pass contextual information downwards, often employing symbol tables to handle nested scopes. [18].

Different programming languages handle type safety in different ways. Statically typed languages like Java check types during compilation, catching errors early but requiring explicit type declarations. Dynamically typed languages like Python delay checks until runtime, offering flexibility at the cost of occasional runtime errors. Additionally, there are also strongly and weakly typed languages. Strong typing ensures that operations are semantically meaningful, for example, preventing the use of undeclared variables, treating a boolean as a function pointer, or attempting to concatenate a string and a number. Whereas weakly typed languages give no such guarantees, and it might silently convert values or interpret bits arbitrarily. [18]

3.2.4 Intermediate Code Generation

Intermediate code generation helps to decouple the front-end (language-specific parsing) from the back-end (machine-specific code generation). During this phase, the compiler translates the abstract syntax tree, produced by the earlier stages, into an intermediate language representation. So, for example, rather than translating directly from high-level languages like Python or Java to specific machine instructions, the compiler first converts the program into an architecture-neutral intermediate form, supporting different hardware architectures.

This step offers practical benefits that make it widely used in compiler design. As Mogensen [18] notes, it makes programs more portable, meaning code compiled to an intermediate form can run on different CPUs without changing the compiler's front-end. This step also simplifies optimization, since improvements like removing dead code only need to be done once for all target machines. Adding new language support also becomes easier, as developers only need to write a front-end that produces the existing intermediate code.

This step introduces some overhead. However, the extra translation step can slow down compilation. Also, designing the intermediate language itself can be tricky. It needs to work with high-level language features while still being efficient for machine code generation. Some compilers use multiple intermediate layers to handle this complexity. [18] Despite these trade-offs, intermediate code remains fundamental to modern compiler design.

3.2.5 Register Allocation Generation

In compiler design, register allocation is a critical step to efficiently utilize the limited set of high-speed CPU registers. This process involves assigning program variables, identified during compilation, to available registers on the target machine. Effective allocation addresses two primary challenges: assignment, the mapping of variables to registers, and spilling - the decision of which variables to temporarily move to slower memory when register resources are exhausted. Register allocation can occur either during intermediate or machine code generation. Performing it earlier supports reuse across multiple target architectures, while postponing it until after machine code generation allows more precise allocation, optimizing register use by combining instructions and eliminating unnecessary temporary variables. [18]

Before register allocation, compilers perform liveness analysis to find out which variables are live at each point in the program. A variable is considered live if its value might be used later, and dead otherwise. This analysis is used to construct an interference graph, where each node represents a variable, and edges indicate that two variables are live at the same time and thus cannot share a register. Graph colouring is then applied to this graph to assign registers, and each colour corresponds to a different register [18]. As shown in Figure 3.2, variables A and C interfere (represented by an edge), meaning they are live at the same point in the program and must be assigned different registers. In contrast, A and D do not interfere, as their live ranges do not overlap, so they can safely share the same register.

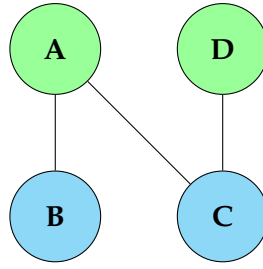


Figure 3.2: Coloured interference graph.

3.2.6 Machine Code Generation

Machine code generation is the phase where intermediate code is translated into architecture-specific assembly instructions. This process makes the platform-independent intermediate language representation into a concrete instruction set of the target CPU (e.g., x86, ARM).

Machine code is the final binary representation of a program, consisting of raw hexadecimal or binary instructions that are entirely hardware dependent. The two most common architectural paradigms are Complex Instruction Set Computer (CISC) and Reduced Instruction Set Computer (RISC). CISC architectures, like x86 processors from Intel and AMD, feature multi-operation instructions and large, highly specialized instruction sets. In contrast, RISC architectures such as ARM processors (used in mobile, IoT devices) prioritize simpler, fixed-width instructions, but may still feature large instruction sets. [3]

For example, the intermediate operation $t = a * b + c$ could be translated into x86 assembly with explicit register allocation and direct memory access to perform multiplication and addition efficiently, as shown below:

```

1 MOV RAX, [a]      ; Load value of 'a' into register RAX
2 IMUL RBX, [b]     ; Multiply RBX by value of 'b'
3 ADD RAX, [c]      ; Add value of 'c' to RAX

```

Figure 3.3: x86 Assembly Translation of $t = a * b + c$

3.3 Alternative Execution Models

3.3.1 Interpreter

An interpreter is an approach that shares many common features with a compiler. Lexing, syntax analysis, and type checking are done in the same way as the compiler. However, instead of generating code from the syntax tree, the syntax tree is processed directly to evaluate expressions and execute statements. Since an interpreter may repeatedly process the same part of the syntax tree, such as a loop body, it is slower than a compiled program. An interpreter is generally used where execution speed is not that important or when each part of the program runs only once. [18]

3.3.2 Transpiler

Transpiler, also called a source-to-source compiler, works by converting one language to another [28]. They are commonly used to bridge outdated codebases to contemporary languages, promoting software longevity and interoperability[21]. Transpilers can convert general-purpose code into domain-specific languages, enhancing efficiency and usability in specialized application domains. [2]

Chapter 4

Design of The Language

In this chapter, we will go through the different design considerations we made for our programming language. We will also go into the syntax and semantics of the language, from the creation of pseudo-code to the implementation of basic syntax, abstract syntax, concrete syntax, and inference rules for the language.

4.1 Language Design Decisions

4.1.1 High Level vs. Low Level Language

We have chosen the high-level language rather than the low-level language for designing our system. The target audience for this system comprises of small-scale game developers who can program at a basic level but do not know low-level memory management and advanced optimizations. Our primary focus is to provide a better user experience, so we have gone for a high-level language that provides a more intuitive and better development experience. High-level language provides greater abstraction and aims to improve developer performance by offering features together with automatic memory management, strong statistics typing and error checking capabilities.

Since we are going to generate a graph for the game, a high-level language does not require more computational resources and provides the best balance between functionality, experience, and maintainability.

4.1.2 Programming Paradigms

Since we wanted to create a programming language that only requires basic programming knowledge and enables the user to quickly define and generate map layouts for game development, we initially chose to go with a declarative approach.

The motivation for choosing the declarative paradigm was that the users of the language would not be required to write down the logic of defining and generating the maps, but simply have to declare them. Having the language handle the backend for defining and generating maps would mean less of a learning curve for users with limited programming

knowledge. Another potential benefit of using a declarative approach is the time required for a user to create a map, which could be minimized by simply declaring the desired structure of the map.

Using a declarative approach seems appropriate for basic map design. However, with the scaling of the language and potential expansion of features to each map element, the paradigm of the language could expand to other paradigms to comply with more complex game mechanics.

4.1.3 Static vs Dynamic Typing

To meet the goals of our language, we have decided to use static typing. Generating maps requires structured data. By implementing static typing, we are forcing developers to use data correctly and avoid bugs that could break the map generation. For example, to create a connection between two rooms, we are strictly enforcing the correct type of "Room" by enabling compile-time error detection. [20]

Additionally, static typing reduces confusion by providing a self-documenting codebase. It allows developers to be more productive and aware of expected types throughout the application development, as a result, providing better readability and maintainability. [20]

4.1.4 DSL vs. GPL

Because our language specifically aims to help with map generation for games, we chose to design it as a domain-specific language (DSL), rather than a general-purpose language (GPL). This specialized focus allows us to design syntax directly tailored to the needs of users working in game development, enhancing readability and usability. Unlike a GPL, our DSL does not support general programming tasks like writing complex algorithms or performing different data manipulation tasks. Instead, we have specialized language features and constructs related to map creation, lowering the learning curve for users.

4.1.5 Compiler vs. Interpreter vs. Transpiler

For our choice of the overall approach to convert our language code into an executable program, we considered three approaches: making a compiler, interpreter, or transpiler. For our language, we decided to go with a compiler. Making a transpiler was discarded since we did not want our language to be dependent on another high-level language's conversion to machine code and any constraints in that regard. For choosing between making a compiler or an interpreter, our main reason for choosing a compiler was the ability to save the machine code output of the compiler. This way, it could be ingrained into other frameworks, for example, a specific game engine, without the need for calling external binaries.

4.2 Syntax and Semantics Design

4.2.1 Pseudo Code

To get a coherent understanding of what we wanted the code to look like in the group, we decided to give some examples of pseudo code. This was done to brainstorm a possible layout of the code and enable better sharing of ideas in the group. We wanted to create code that was relatively simple and easy to read to cater to our target group. This process ended with the following example of some potential pseudo code:

```
1 Map map (maxRooms = 4; maxConnectPerRoom = 3;){
2   Room A;
3   Room B;
4   Connect(A -> B);
5   Room C;
6   Connect (C <-> B);
7 }
```

Figure 4.1: A pseudo code example for the language

4.2.2 Basic Syntax Rules

After agreeing on the pseudo code, we started creating some basic syntax rules for the base case of a simple graph map. The process started by defining the elements we would need to mimic the structure of the pseudo code example. This resulted in the following syntax rules:

Keywords: We decided to implement some keywords that have a predefined meaning of the different elements in a graph map. Firstly, we have "Map", which essentially is a keyword for one map. Then the keyword "Room" is used to describe a fraction of the map, such as one node in a graph map. Finally, the keyword "Connect" describes the connection between each "Room" in the "Map". The naming of the words is based on that the language is defined for general map generation and not just graph maps like the base case.

Identifiers: Each keyword would also be assigned an identifier for distinction between elements in the program. The identifier has to start with an upper or lowercase letter and can, for example, not start with a number.

Operators: The language should also support some operators. For the connection between "Rooms", we decided to use "->" (directional connection) and "<->" (bidirectional connection). These were chosen to cater to our target group of less experienced programmers. The idea is to use the real-life concept of drawing arrows between objects to illustrate a connection.

Map Constraints: Another thing we wanted was for users of the language to be able to tailor the map further than just the previously stated room connections. Therefore, we decided to add two types of map constraints. The first one would constrain the map to have a max allowed number of rooms. The second one would constrain each room to have a maximum allowed number of connections to other rooms.

General Syntax Rules: We also set up some general rules for how code is set up: After each line of code, we use punctuation in the form of a semicolon, and for the block structure of the code, we use curly brackets instead of being sensitive to indentation.

4.2.3 The Abstract Syntax

After having defined what we wanted the basic syntax rules of the language to be, we moved on to creating the abstract syntax for the language to define the building blocks that would make up the language. In figure 4.2, it can be seen what the formation rules of the abstract syntax look like.

$$\begin{aligned}
 P &\in \text{Program} ::= \mathbf{Map} \text{ id MapConstr } \{S^*\} \\
 \text{MapConstr} &\in \text{MapConstraints} ::= \epsilon \mid (\text{ConstrList?}) \\
 \text{ConstrList} &\in \text{ConstraintList} ::= \text{Constr} \mid \text{Constr}; \text{Constr}; \\
 \text{Constr} &\in \text{Constraint} ::= \mathbf{maxRooms} = n \mid \mathbf{maxConnectPerRoom} = n \\
 S &\in \text{Statement} ::= R \mid C \\
 R &\in \text{Room} ::= \mathbf{Room} \text{ id}; \\
 C &\in \text{Connection} ::= \mathbf{Connect} (\text{id Op id}); \\
 \text{Op} &\in \text{Operator} ::= \rightarrow \mid \leftrightarrow \\
 \text{id} &\in \text{Identifier} \\
 n &\in \text{Integer}
 \end{aligned}$$

Figure 4.2: The updated formation rules for the abstract syntax of the language, with optional map constraints

The abstract syntax consists of 10 syntactic categories [18], where each element of a specific syntactic category has the same behaviour.

The syntactic category *Program* has one formation rule, which states that you have to use the **Map** keyword followed by an *id*, which would be something of the *Identifier* syntactic category. After this, you have something of the *MapConstraints* syntactic category followed by any amount (denoted by ***) of elements from the *Statement* syntactic category, which would then be placed within curly brackets.

An element from the *Statement* syntactic category can either be of the *Room* syntactic category or the *Connection* syntactic category. Elements of the *Room* syntactic category are created using the **Room** keyword followed by an *id* of the *Identifier* syntactic category, and finally, a semi-colon is used to finish the element.

To create an element of the *Connection* syntactic category, the **Connect** keyword is first used, followed by a starting parenthesis, an *id* of the *Identifier* syntactic category, followed by an operator of the *Operator* syntactic category. The *Operator* syntactic category has two different formation rules where an element of the category can either look like " $\rightarrow$ " or " $\leftrightarrow$ ". Finally, another element of the *Identifier* category is used along with an ending parenthesis, and like *Room* elements, a semi-colon finalizes the element.

For the *MapConstr* used within the formation rule of the *Program* syntactic category, we had to set up some underlying syntactic categories to make the correct logic behind how map constraints would work. First, the *MapConstraints* syntactic category has two formation rules. It can either be empty (denoted by ϵ), meaning there are no map constraints assigned, or it can be 0 or 1 element (denoted by *?*) of the *ConstraintList* syntactic category

placed within parenthesis. If it is something of the *ConstraintList* syntactic category, it can be either a single constraint (*Constr*) or two constraints separated by a semicolon (*Constr;Constr*). An element of the *Constraint* syntactic category can either be the **maxRooms** constraint or the **maxConnectPerRoom** constraint, and either way, it would have to be equal to an integer (denoted by *n*).

Using the defined abstract syntax, we can consider an element of a syntactic category as something that can be represented using a tree-like structure, where the nodes in the tree will be made up of instances of formation rules. This tree-like structure is called an "Abstract Syntax Tree" and in the following example, it can be seen how one such tree would look for an element of the "Program" syntactic category defined as:

Map myMap (maxRooms = 4) {Room A; Room B; Connect (A -> B);}

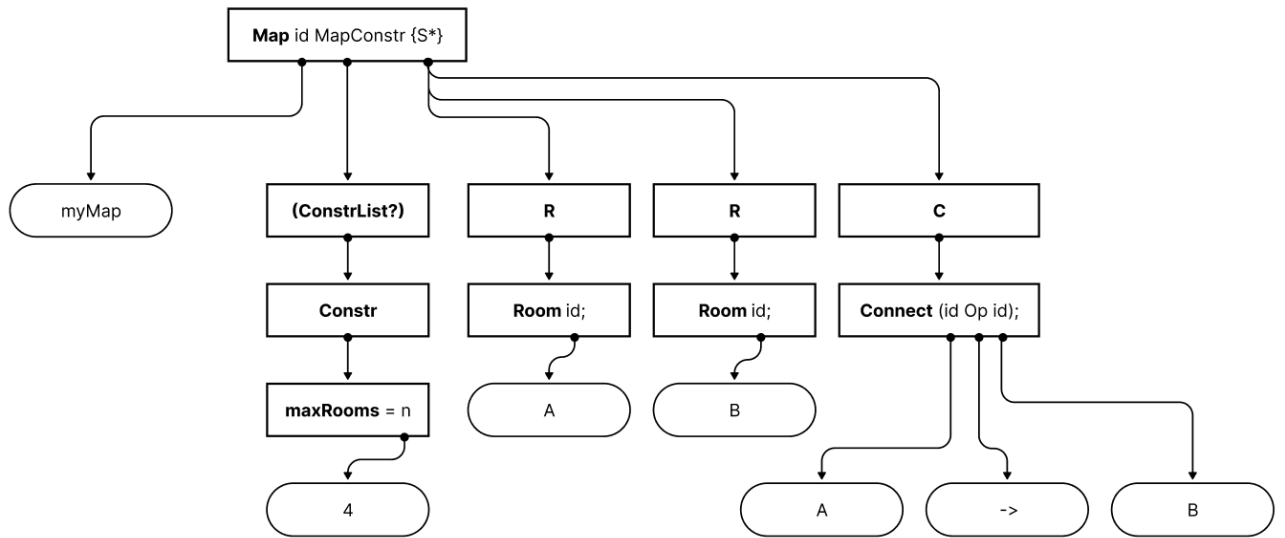


Figure 4.3: An example of an abstract syntax tree based on an element of the "Program" syntactic category.

4.2.4 The Concrete Syntax

Concrete syntax defines the precise textual representation of a programming language, which developers will use to write and read. It determines the specific grammatical rules under which programs can be physically represented using characters, symbols, and keywords. Such as parentheses, braces, operators, and reserved words.

Concrete syntax is used for writing, parsing, and displaying code and will often correspond exactly to the abstract syntax trees during compilation. Therefore, it must be simple, consistent to support easy parsing and tool development. However, while enabling human interaction with the code, it may sometimes be verbose or less intuitive, especially in cases where there is a lack of visual or mathematical formatting at times. [18]

$$\begin{aligned}
P &\in \text{Program} ::= \text{"Map"} \text{ id } \text{MapConstr} \text{"{" } S^* \text{"}" } \\
\text{MapConstr} &\in \text{MapConstraints} ::= \epsilon \mid \text{"(" } \text{ConstrList?} \text{"}" } \\
\text{Constr} &\in \text{ConstraintList} ::= \text{Constr} \mid \text{Constr} \text{";" } \text{Constr} \text{";" } \\
\text{Constr} &\in \text{Constraint} ::= \text{"maxRooms"} \text{"=" } n \mid \text{"maxConnectPerRoom"} \text{"=" } n \\
S &\in \text{Statement} ::= R \mid C \\
R &\in \text{Room} ::= \text{"Room"} \text{ id } \text{";" } \\
C &\in \text{Connection} ::= \text{"Connect"} \text{"(" } \text{id } \text{Op } \text{id} \text{"}" } \text{";" } \\
\text{Op} &\in \text{Operator} ::= \text{"->" } \mid \text{"<->" } \\
\text{id} &\in \text{Identifier} ::= \text{Letter} (\text{Letter} \mid \text{Digit})^* \\
n &\in \text{Integer} ::= \text{Digit}^* \\
\text{Letter} &::= \text{"a"}\text{"-"}\text{"z"} \mid \text{"A"}\text{"-"}\text{"Z"} \\
\text{Digit} &::= \text{"0"}\text{"-"}\text{"9"}
\end{aligned}$$

Figure 4.4: The formation rules for the concrete syntax of the language

4.2.5 Inference Rules for Static Semantics

Static semantics are the compile-time checks that ensure a program is well-formed. It specifies correctness conditions that a program must meet beyond syntactic correctness. [19] For our DSL, we used inference rules to express and formalize them.

In our system, we define two environments:

- Γ : the symbol table, a function mapping room identifiers to their number of current connections:

$$\Gamma : \text{Id} \rightarrow \mathbb{N}$$

- Σ : the constraint environment, a function mapping constraint names to natural numbers:

$$\Sigma : \{\text{maxRooms}, \text{maxConnectPerRoom}\} \rightarrow \mathbb{N}$$

Map declaration:

The overall program is a map declaration that introduces a new identifier to the type environment of type `Map`, as well as combining a sequence of statements S^* , that can be constrained by optional `MapConstr`:

$$\frac{\text{MapConstr} \Rightarrow \Sigma \quad \Sigma_0 \Gamma_0 \quad \Sigma \vdash S^*}{\text{Map id (MapConstr) } \{ S^* \}}$$

The above premise explains that if the constraint block `MapConstr` is well-formed and yields a constraint environment Σ , and if the statement sequence S^* is valid under Σ , then the map block is considered well-formed. Initially, both environments are empty: Γ_0 represents the empty symbol table, and Σ_0 is produced from constraint declarations.

Constraint declaration:

We can define two constraint rules for the map - `maxRooms` and `maxConnectPerRoom`:

$$\frac{n_1 \in \mathbb{N} \quad n_2 \in \mathbb{N}}{\text{maxRooms} = n_1; \text{maxConnectPerRoom} = n_2 \Rightarrow \Sigma = \{(\text{maxRooms}, n_1), (\text{maxConnectPerRoom}, n_2)\}}$$

This defines Σ as a mapping from constraint names to their corresponding natural number values.

Room declaration:

A room declaration introduces a new identifier into the type environment and can only be added if it hasnt been declared yet and the total number of rooms does not exceed the maximum specified by the constraint environment Σ . The room is initialized with zero connections. We formalize this as:

$$\frac{id \notin \Gamma \quad |\Gamma| + 1 \leq \Sigma(\text{maxRooms})}{\Sigma, \Gamma \vdash \text{Room } id; \Rightarrow \Gamma' = \Gamma[id \mapsto 0]}$$

Connection declaration:

A connection is only valid if both room ids exist in the environment Γ and each has fewer than `maxConnectPerRoom` current connections. Once validated, the rule increments the connection count for both rooms:

$$\frac{id_1, id_2 \in \Gamma \quad \Gamma(id_1) + 1, \Gamma(id_2) + 1 \leq \Sigma(\text{maxConnectPerRoom})}{\Sigma, \Gamma \vdash \text{Connect } (id_1 \text{ Op } id_2); \Rightarrow \Gamma' = \Gamma[id_1 \mapsto \Gamma(id_1) + 1, id_2 \mapsto \Gamma(id_2) + 1]}$$

Sequence composition:

This rule is intended to capture how a single statement S and a sequence of statements S^* combine to form a larger sequence:

$$\frac{\Sigma \vdash S_1 \Rightarrow \Gamma' \quad \Sigma \vdash S_2^* \Rightarrow \Gamma''}{\Sigma, \Gamma \vdash S_1; S_2^* \Rightarrow \Gamma''}$$

Statement sequencing is valid if the first statement S_1 is valid under Σ and yields an updated environment Γ' , and if the remaining statements S_2^* are also valid under Σ starting from Γ' and yield Γ'' .

Together, these inference rules form the foundation of our static type system, ensuring that programs written in our DSL are structurally sound and free of semantic inconsistencies.

Chapter 5

Implementation

In this chapter, each step of the compiler pipeline will be explained in depth. The explanation of the individual components will also include a demonstration of the pseudo code presented in section 4.2.1 being inserted into each component. Our compiler goes through five steps:

- **Lexer**
- **Parser**
- **Semantic Analysis**
- **Intermediate Language (IL)**
- **Code Generation**

The frontend phases, from Lexer to Semantic Analysis, contain a global error handling framework, described in section 5.1.3. This system keeps error catching and feedback tied to the compilers front-end design. After that, the backend takes over, transforming the source code into machine code through the use of an intermediate language (IL), as well as a combination of handwritten and generated assembly code. See Figure 5.1 for a visual representation of the entire compiler pipeline.

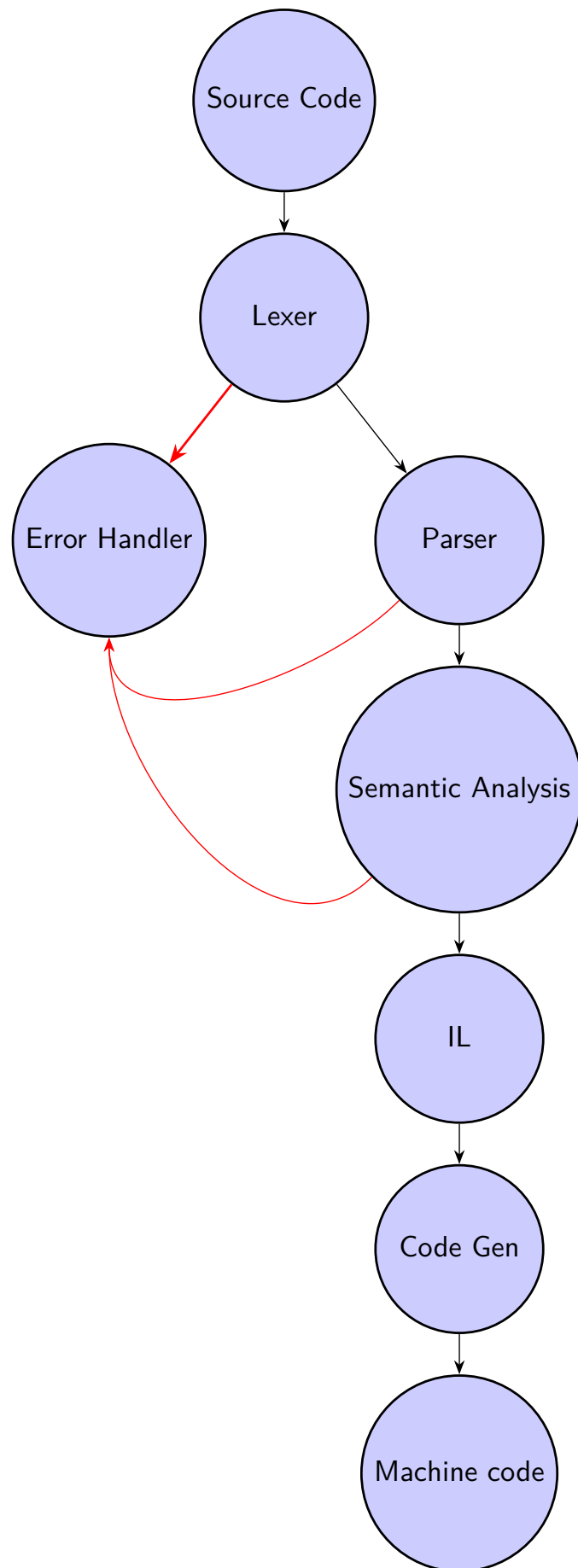


Figure 5.1: Visual representation of the compiler pipeline

5.1 Common

In the front-end of the compiler, several components share functionalities and definitions. Therefore we made a folder containing functions and datatypes shared by the front-end components, mainly to simplify the code base and to avoid having duplicate implementations. These shared utilities are organized into different components:

- **Compiler state** manages lexer information, file utilities and source code location.
- **Definitions** is used to store constants and types, such as token definitions and abstract syntax tree structures.
- **Error handler** a global framework to handle error messages.

5.1.1 Compiler State

This component is responsible for managing lexer-related information. It maintains the state of the input source file, including whether the file is open, the current character position and if a character needs to be putback. It also provides functions for file handling throughout the compiler, as well as a location tracking algorithm used by the error handler (see section 5.1.3). Some important helper functions for this component are:

Open Input File: This function tries to open a file. If the file isn't found, it calls `exit` and kills the program. If it succeeds, it stores the file in `infile` and resets the lexer info to its default starting values.

Converts File Position Into a Line and Column Number: This function takes a character position in a file and converts it into a line and column number. It initializes a custom data structure called `CLoc` (Compiler Location) and checks if the source code file is open. If no file is open, it returns a default location 1, 1. Then, it rewinds to the start of the source code to ensure it's at the first character. After that, it processes characters up to the given position, using a switch statement to increment the line number, column number, or reset the column based on the symbol encountered until it reaches the target position.

```

1 CLoc findLoc(int pos)
2 {
3     CLoc loc = {1, 1};
4     if (!cs.infile || pos < 0)
5         return loc;
6
7     rewind(cs.infile);
8     for (int i = 0; i < pos; i++)
9     {
10         int c = fgetc(cs.infile);
11         switch (c)
12         {
13             case EOF: break;
14
15             case '\n':
16                 ++loc.line;
17                 loc.column = 1;
18                 break;
19
20             case '\t': loc.column += 4 - (loc.column - 1) % 4; break;
21
22             default: ++loc.column;
23         }
24     }
25     return loc;
26 }
27

```

Figure 5.2: Source code location finder

5.1.2 Definitions

This component contains definitions of constants and types used in our compiler. For the current state of the compiler it contains two constants and two data types. The first constant limits the input string to a maximum of 32 characters, helping to ensure safe memory management and follow the size of CPU alignments. The other constant is an integer value that we use to set maximum allowed connections per room explained in the semantic analysis (see section 5.5). The first data type `TokenDef` defines all possible types of tokens that consist of keywords, identifiers, numbers and delimiters. The last data type `AbstractTokenDef` defines types representing nodes in the Abstract Syntax Tree.

```

1 typedef enum {
2     T_INTEGER,           // int
3     T_FLOAT,             // float
4     T_MAP,               // Map
5     ...,
6 } TokenDef;
7
8 typedef enum {
9     AT_MAP = T_MAP,       // Map
10    AT_ROOM = T_ROOM,     // Room
11    AT_CONNECT = T_CONNECT, // Connect
12    AT_DIRECTED_EDGE = T_DIRECTED_EDGE, // ->
13    ...,
14 } AbstractTokenDef;
15

```

Figure 5.3: A snippet of the two token types

5.1.3 Error Handler

The error handler is responsible for reporting error messages directly to the terminal. It includes a data type that encapsulates error codes, with distinct values assigned to each compiler phase. A unified `reportError` function is used across front-end components to provide consistent error reporting, including the source code location. Additionally, the handler has a `testMode` flag and a toggle function: when enabled, errors are returned as codes instead of terminating the compiler, allowing for testing.

line 1-2: The `reportError` function takes an error code, a position (`pos`), and a variable number of strings used to form the error message. The argument `msgc` specifies how many strings are passed. If `testMode` is enabled, the function immediately returns the error code instead of terminating.

line 4-11: The function initialize message string, and then safely concatenate multiple strings using a using a helper function `safe_multi_strcat` that checks if the concatenated strings are still inside the error length limit. if inside the the limit the function will, return a custom error message with source code location using the `findLoc` function from the compiler state module (see Section 5.1.1).

```

1 int reportError(ErrorCode code, int pos, const char *strings[], size_t
  msgc) {
2     if(isTestMode) return code;
3
4     char msg[MAX_ERROR_LENGTH] = {0};
5     int status = safe_multi_strcat(msg, strings, msgc, MAX_ERROR_LENGTH);
6     CLoc loc = findLoc(pos);
7     if(status == -1){
8         fprintf(stderr, "Failed to build the error message");
9     }
10
11     fprintf(stderr, "%d:%d: %s %s\n", loc.line, loc.column, ERROR_LABELS[
      code], msg);
12
13     exit(1);
14 }
15

```

Figure 5.4: Error reporter

5.2 Lexical Analyzer

Lexical analysis is used to tokenize a high-level language in a form that can be processed further. We are converting the source characters into tokens, which are symbols like keywords, identifiers, numbers and delimiters that will be passed on to the parser.

Token Structure: Token structure is used to store information about each token identified by lexer and contains:

token: enum value representing token type from TokenDef. (See figure 5.3)

pos: start position of a token in the input file.

value: the string content of a token. It is essential for storing values like identifier names or numbers. MAX_INPUT_SIZE is a constant number value that currently restricts the string value to a maximum of 32 characters.

```

1 typedef struct Token {
2     TokenDef token;
3     int pos;
4     char value[MAX_INPUT_SIZE];
5 } Token;
6

```

Figure 5.5: Token structure

Initializing a Token: The lexical analysis component uses a helper function that initializes Token Structure by assigning its type, starting position and value. In cases where the token type is EOF(end of file), it assigns an empty string as a value.

```

1 int set_token(Token* t, TokenDef type, const char* value, int pos)
2 {
3     if (type == T_EOF) {
4         if (!t) return 0;
5         t->token    = T_EOF;
6         t->pos      = pos;
7         t->value[0] = '\0';
8         return 1;
9     }
10
11     if (!t || !value || *value == '\0')
12         return 0;
13
14     t->token    = type;
15     t->pos      = pos;
16     int status = safe_strcpy(t->value, value, MAX_INPUT_SIZE);
17
18     return status;
19 }
20

```

Figure 5.6: Function to initialize a token with type, value, and position

Token Map Matching: To find whether a key string is part of a provided token map the component uses a helper function `match_map`.

Line 1-4: the function takes a token map `text map[]`, a string key `key`, token position `pos`, The token we want to initialize `token`, and the size of the given token map `size`. The function starts by doing a safety check to see if given a key or token, if not return `NULL`.

Line 5-13: The function iterates through the token map and compares the given string key with the token names. If the key matches a token name, the given token is initialized, using the `set_token`, and the function returns a boolean as a status.

```

1 int match_map(const Map map[], const char* key, const int pos, Token*
   token, const size_t size)
2 {
3     if (key == NULL || token == NULL)
4         return 0;
5     for (int i = 0; i < size; i++)
6     {
7         if (strcmp(key, map[i].name) == 0)
8         {
9             set_token(token, map[i].token, map[i].name, pos);
10            return 1;
11        }
12    }
13    return 0;
14 }
15

```

Figure 5.7: Function to match a string against a token map

Putting Back a Character:

The component uses a helper function to put a character back into the input stream. It is used when lexer reads beyond the end of a token and needs to put back a character, so it can be reprocessed correctly. It will store a put back character "c" in a compiler state and remove input file position by one, so we can read the character again correctly.

```
1 static void putback(int c)
2 {
3     cs.putback = c;
4     cs.pos--;
5 }
6
```

Figure 5.8: Function to put a character back into the input stream

Reading Next Character:

The component uses a helper function to consume characters from the input file. It checks if a character was previously put back. If so, it will set the putback to 0 and return the putback character. This ensures that the character is only reused once. If no character was put back, it will read the next character from the input file, update input file counter position, and return newly read character.

```
1 static int next(void)
2 {
3     if (cs.putback)
4     {
5         int c = cs.putback;
6         cs.putback = 0;
7         return c;
8     }
9
10    int c = fgetc(cs.infile);
11
12    cs.pos++;
13
14    return c;
15 }
16
```

Figure 5.9: Function to read the next character from the input

Whitespace Skipping:

The component uses a helper function to read and ignore whitespace characters from the input stream until it reaches the first non-whitespace character and returns it. This cleans up the input stream before trying to find a new token to simplify tokenization logic.

```

1 static int skip(void)
2 {
3     int c;
4     while ((c = next()) && isspace(c)) {}
5     return c;
6 }
7

```

Figure 5.10: Function to skip whitespace and return first character

Tokenizing Input Characters:

The component uses a helper function to read characters from the input stream until it constructs a valid token based on lexical rules. It classifies token types into integers, identifiers, delimiters, keywords and reports an error if token is invalid.

line 1-5: The function takes a token *t*, the first read character after removing whitespace *c* and a counter position in the source code *startPos*. Then the function starts by declaring three values the base value for the loop iteration *i* at 0, the buffer that is the token string value which is build as the function read trough each character and then the *isIdentifier* which work as a boolean indicating whether an identifier has already been found.

line 7-16: The function checks if the given read character is a digit and if so it will loop trough the next characters as long as its still digits and is under the max input size limit. when the while loop ends the function adds a '\0' to the buffer. This is done as C relies on it to know where the string ends. Then the function initialize the token with appropriate values and puts back a character as last character is never a digit and afterwards it returns the token.

```

1 int tokenize(Token* t, int c, int startPos)
2 {
3     int i = 0;
4     char buffer[MAX_INPUT_SIZE] = {0};
5     int isIdentifier = 1;
6
7     if (isdigit(c)) {
8         while (isdigit(c) && i < MAX_INPUT_SIZE - 1) {
9             buffer[i++] = c;
10            c = next();
11        }
12        buffer[i] = '\0';
13
14        set_token(t, T_INTEGER, buffer, startPos);
15        putback(c);
16        return t->token;
17    }
18
19    ...
20 }
21

```

Figure 5.11: Handling of numeric tokens

If the first character was not a digit, the function enters a loop to collect characters until it reaches whitespace or a delimiter. As it enters the loop, it will store the first character in a buffer, if its not whitespace. Afterwards, it will check if this character is invalid for an identifier, if true, it will set `isIdentifier` boolean to false. Then it checks if it is a delimiter and if true, it breaks out of the loop, if not, it reads the next character and if next character is a delimiter, this means we are going beyond the current token, so it puts back a character and breaks out of the loop, so it can handle delimiter as next token. However, if it did not yet hit whitespace or identifier, it will continue looping until we take all the characters for the possible token.

```

1 int tokenize(Token* t, int c, int startPos)
2 {
3     int i                = 0;
4     char buffer[MAX_INPUT_SIZE] = {0};
5     int isIdentifier      = 1;
6
7     ...
8
9     while (!isspace(c) && i < MAX_INPUT_SIZE - 1)
10    {
11        buffer[i++] = c;
12
13        if (!(isalnum(c) || c == '_' || c == '.') && !isspace(c))
14            isIdentifier = 0;
15
16        if (is_delimiter(c))
17            break;
18
19        c = next();
20
21        if (is_delimiter(c))
22        {
23            putback(c);
24            break;
25        }
26    }
27
28    buffer[i] = '\0';
29
30    ...
31 }
32

```

Figure 5.12: Loop to collect characters into buffer

Lastly, after it finished collecting buffer string for a possible token, it will determine what kind of token has been collected. If the last character read is of type end of file, token type will be set to EOF. If the buffer matches delimiter or keyword, it will call lookup function which will set the token accordingly. If the buffer passed identifier check, it will set the token to an identifier and put back a character because it's not part of an identifier. In all instances, if the token has been found, it will be returned, otherwise, an error is reported using `reportError`.

```

1 int tokenize(Token* t, int c, int startPos)
2 {
3     int i                = 0;
4     char buffer[MAX_INPUT_SIZE] = {0};
5     int isIdentifier      = 1;
6
7     ...
8
9     if (c == EOF)
10    {
11        set_token(t, T_EOF, NULL, startPos);
12        return t->token;
13    }
14
15    if (lookup_delimiter(buffer, startPos, t))
16        return t->token;
17
18    if (lookup_keyword(buffer, startPos, t))
19        return t->token;
20
21    if (isIdentifier)
22    {
23        set_token(t, T_IDENTIFIER, buffer, startPos);
24        putback(c);
25        return t->token;
26    }
27
28    const char* msg[] = {"Invalid token: '", buffer, "' "};
29    return reportError(ERR_LEXER, startPos, msg, 3);
30 }
31

```

Figure 5.13: Token classification and error handling

Reading Next Token:

The main function of lexer, it reads and classifies the next token from an input file.

First, it checks if the file is open and then it will initialize a variable `c` with the first character after skipping all the whitespace behind it. It will also initialize `startPos` with current input file's counter position and pass all the necessary information into `tokenize` function to find the next token.

```

1 int scan(Token* t)
2 {
3     if (!csIsFileOpen())
4     {
5         perror("Lexer could not find input");
6         return -1;
7     }
8
9     int c      = skip();
10    int startPos = cs.pos;
11    return tokenize(t, c, startPos);
12 }
13

```

Figure 5.14: Main scan function to initiate tokenization

Tokenization Process for Pseudo Code: In Figure 5.15 it can be seen what the lexer would output for the pseudo code example.

```

[T_MAP("Map"), T_IDENTIFIER("map"), T_LPAREN("("), T_MAP_CONSTR_ROOMS("
maxRooms"), T_EQUAL("="), T_INTEGER("4"), T_SEMICOLON(";"),
T_MAP_CONSTR_CONNECT("maxConnectPerRoom"), T_EQUAL("="), T_INTEGER("3"),
T_SEMICOLON(";"), T_RPAREN(")"), T_LBRACE("{"), T_ROOM("Room"),
T_IDENTIFIER("A"), T_SEMICOLON(";"), T_ROOM("Room"), T_IDENTIFIER("B"),
T_SEMICOLON(";"), T_CONNECT("Connect"), T_LPAREN("("), T_IDENTIFIER("A"),
T_DIRECTED_EDGE("->"), T_IDENTIFIER("B"), T_RPAREN(")"), T_SEMICOLON(";"),
T_ROOM("Room"), T_IDENTIFIER("C"), T_SEMICOLON(";"), T_CONNECT("Connect"),
T_LPAREN("("), T_IDENTIFIER("C"), T_BIDIRECTIONAL_EDGE("<->"),
T_IDENTIFIER("B"), T_RPAREN(")"), T_SEMICOLON(";"), T_RBRACE("}")]

```

Figure 5.15: The lexer's output for the pseudo code example

5.3 Abstract Syntax Tree Implementation

Before the parsing part of the language we have the underlying functions to create the abstract syntax tree which will be utilized by the parser. The different underlying elements of the abstract syntax tree we refer to as nodes of the tree and these are defined using a struct called `ASTNode`. This struct consists of information about the node's type (for example a map node or a room node), the node's position and information related to its child nodes.

```

1 typedef struct ASTNode {
2     AbstractTokenDef type;
3     int pos;
4     char data[MAX_INPUT_SIZE];
5     struct ASTNode** children;
6     int child_count;
7     int child_capacity;
8 } ASTNode;
9

```

Figure 5.16: The ASTNode struct

Initializing the Abstract Syntax Tree: The first function we have is a function to initialize the abstract syntax tree. What is mainly done here is assigning a pointer to the tree, which we will be utilizing throughout the other functions.

```

1 ASTree ASTInit()
2 {
3     ASTree tree = calloc(1, sizeof(*tree));
4     if (tree == NULL)
5         return NULL;
6 }
7     tree->head = NULL;
8     return tree;
9 }
10

```

Figure 5.17: The ASTInit function

Adding child nodes: The functions for the creation of the abstract syntax tree utilize the `ASTAddChild` and `ASTResizeChildren` functions to add new child nodes to existing nodes or update a given nodes child capacity, respectively.

`ASTAddChild` works by first checking that we have both a parent node and a child node to add to it. If this is the case it then checks that there is space for the child on the parent node, meaning that the amount of child nodes on the parent node do not exceed the capacity of children the parent node can currently have. If there is space then the child node is simply added to the parent's array of children and the `child_count` value of the parent is incremented by one.

`ASTResizeChildren` is utilized when we need a parent node to have space for more child nodes. It works by assigning a new value for `child_capacity` and adding the existing children to a new array with the new capacity.

The creation of different nodes: All the functions for creating different types of nodes share some common elements. They all utilize the pointer created earlier to allocate memory for the respective node they are about to create. After this, they set some default values for the elements of the `ASTNode` struct:

- `type` is set to the appropriate enum value of the node.
- `child_count` is initially set to 0 since no nodes have children yet and some nodes might not end up having children.
- `child_capacity` is mostly set to 0 but for the nodes where the amount of children it will have is already known the value is instead set to the value of known children.
 - For the Room node, it is set to 1 since it will have its identifier as a child node.
 - For the Map Constraint node, it is set to 1 since it will have its value as a child node.
 - For the Connect node, it is set to 3 since it will have child nodes for the two room identifiers and one for the connect operator.

On figure 5.18 an example of how one of the node creation functions work, in this case the one for creating a Room node.

line 1-4: The function takes a parent node `mapNode`, an id `id` and a position `pos`. Then the function first do a safety check to see if actually given an node.

Line 6-11: Here the function checks if the number of nodes in the AST has reached the capacity and if so it will increase capacity and allocate more space. After that the function will initialize a new AST node.

Line 13-19: Here the different variables associated to the new AST node is declared.

Line 20-28: The function allocates memory for a child node and do a safety check on the allocation was successful. Then on line 28 it add a child node to the new AST node, in this case the child node of a Room type is the identifier.

line 29: The new AST node created in the function is now added as a child node to the parent node `mapNode`

```

1 void ASTCreateRoom(ASTNode* mapNode, const char* id, int pos)
2 {
3     if (!mapNode)
4         return;
5
6     if (mapNode->child_count >= mapNode->child_capacity)
7         ASTResizeChildren(mapNode);
8
9     ASTNode* roomNode = calloc(1, sizeof(ASTNode));
10    if (roomNode == NULL)
11        return;
12
13    roomNode->type = AT_ROOM;
14    safe_strcpy(roomNode->data, "Room", MAX_INPUT_SIZE - 1);
15
16    int child_capacity      = 1;
17    roomNode->pos           = pos;
18    roomNode->child_count   = 0;
19    roomNode->child_capacity = 1;
20    roomNode->children      = calloc(child_capacity, sizeof(ASTNode*));
21
22    if (roomNode->children == NULL)
23    {
24        free(roomNode);
25        return;
26    }
27
28    ASTAddChild(roomNode, ASTCreateIdentifier(id));
29    ASTAddChild(mapNode, roomNode);
30 }
31

```

Figure 5.18: The ASTCreateRoom function

5.4 Parser

After having created the framework for building the abstract syntax tree we moved on to produce the code for the rest of what would be needed for our parser. To aid our parsing process we first created two helper functions: `consumeToken` and `errorHelper`.

Consuming tokens: The `consumeToken` function is used to consume the current token and update to the next one if the current token matches the expected token's type. It works by first checking whether the current token is equal to `NULL`. If this is the case, an error message is displayed, otherwise it moves on to check if the current token is equal to the expected one. If this is correct it updates the current token to the next one using the lexer and returns 1 to state that it successfully consumed the token we had. In case of an unexpected token it returns 0 to state that it failed to consume the token.


```

1 int consumeToken(Token* currentToken, TokenDef expectedTokenType)
2 {
3     if (currentToken == NULL)
4     {
5         fprintf(stderr, "Expected a token but got NULL");
6     }
7
8     if (currentToken->token == expectedTokenType)
9     {
10        scan(currentToken);
11        return 1;
12    }
13
14    return 0;
15 }
16

```

Figure 5.19: The consumeToken function

Custom error messaging: The other helper function, called `errorHelper`, uses the `reportParseError` function from the `error_handler` file (see section 5.1.3) to help create error messages when we don't have the expected token.

Along with the two helper functions, we created a variety of different functions to check if the correct tokens were used at different points throughout an expression, and if yes, used the corresponding function from our `astree` file (see section 5.3) to build the specific part of the abstract syntax tree.

Checking Map expressions: The `mapExpr` function checks that we have the correct token structure of an Map element.

Line 1-6: It first gets the first token in the map and utilizes the `consumeToken` function to check if the current token given by the lexer matches that of a Map keyword token. If this is not the case, an error message is given using the `errorHelper` function. For a correct token it instead moves on to check the next token.

Line 8-13: Since the next token is an id that we would like to store for later use, we make sure to use `strdup` to create a copy and this way make sure that we have a copy, even after `consumeToken` later tells the lexer to move on to the next token. Next step is checking that the current token we have actually is an identifier token. If this is the case we can now use the `ASTCreateMap` function from the `astree` file (see section 5.3) to start building the Map part of the abstract syntax tree.

Line 15-19: After this we check whether we have any map constraints which would be indicated by having a left parenthesis after the map identifier. If this is the case we run the `parseMapConstr` function to do the necessary steps of checking the map constraint tokens and building the map constraints part of the abstract syntax tree.

Line 21-29: Finally we check that we have the correct tokens for the starting and ending curly-brackets, and in between this we use the `parseMapBody` before finally returning a `mapNode` for later use.

```

1 ASTNode* mapExpr(ASTree tree, Token* currentToken)
2 {
3     int mapPos = currentToken->pos;
4
5     if (!consumeToken(currentToken, T_MAP))
6         reportParserError("Map", currentToken);
7
8     const char* id = strdup(currentToken->value);
9
10    if (!consumeToken(currentToken, T_IDENTIFIER))
11        reportParserError("identifier", currentToken);
12
13    ASTNode* mapNode = ASTCreateMap(tree, id, mapPos);
14
15    if (consumeToken(currentToken, T_LPAREN)) {
16        parseMapConstr(mapNode, currentToken, id);
17        if (!consumeToken(currentToken, T_RPAREN))
18            reportParserError(")", currentToken);
19    }
20
21    if (!consumeToken(currentToken, T_LBRACE))
22        reportParserError("{", currentToken);
23
24    parseMapBody(mapNode, currentToken);
25
26    if (!consumeToken(currentToken, T_RBRACE))
27        reportParserError("}", currentToken);
28
29    return mapNode;
30 }
31

```

Figure 5.20: The mapExpr function

Checking the Map Constraints: Given that the `parseMapConstr` function is called after having checked for a left parenthesis token it immediately starts out by checking for a right parenthesis token since this would mean the map constraints are empty and we would simply be able to return. If this is not the case the function then sees whether the next tokens match that of the `MaxRooms` constraint or the `MaxConnectPerRoom`. Depending on which constraint the next token match the function runs through a collection of `consumeToken` function calls to check the individual parts of the constraint for correct occurrence of tokens and if everything succeeds the map constraint is added to the abstract syntax tree with the help of `ASTCreateMapConstr` (see section 5.3).

```

1 void parseMapConstr(ASTNode* mapNode, Token* currentToken, const char* id
  ){
2     if (currentToken->token == T_RPAREN)
3         return;
4
5     int mapNodePos = currentToken->pos;
6
7     if(consumeToken(currentToken, T_MAP_CONSTR_ROOMS)){
8         if(!consumeToken(currentToken, T_EQUAL))
9             reportParserError("=", currentToken);
10
11         char* constr_value = strdup(currentToken->value);
12         if(!consumeToken(currentToken, T_INTEGER))
13             reportParserError("an Integer", currentToken);
14
15         if(!consumeToken(currentToken, T_SEMICOLON))
16             reportParserError(";", currentToken);
17
18         char idConstr[MAX_INPUT_SIZE];
19         snprintf(idConstr, sizeof(idConstr), "%sMaxRooms", id);
20
21         ASTCreateMapConstr(mapNode, "MaxRooms", mapNodePos,
22             AT_MAP_CONSTR_ROOMS, constr_value);
23     } else if(consumeToken(currentToken, T_MAP_CONSTR_CONNECT)){
24         if(!consumeToken(currentToken, T_EQUAL))
25             reportParserError("=", currentToken);
26
27         char* constr_value = strdup(currentToken->value);
28         if(!consumeToken(currentToken, T_INTEGER))
29             reportParserError("an Integer", currentToken);
30
31         if(!consumeToken(currentToken, T_SEMICOLON))
32             reportParserError(";", currentToken);
33
34         char idConstr[MAX_INPUT_SIZE];
35         snprintf(idConstr, sizeof(idConstr), "%sMaxConnectPerRoom", id);
36         ASTCreateMapConstr(mapNode, "MaxConnectPerRoom", mapNodePos,
37             AT_MAP_CONSTR_CONNECT, constr_value);
38     } else {
39         reportParserError("maxRooms | maxConnectPerRoom", currentToken);
40     };
41
42     parseMapConstr(mapNode, currentToken, id);
43 }
44

```

Figure 5.21: The parseMapConstr function

Checking for Room or Connection expressions: The parseMapBody function is used to recursively check and add the Room and Connect elements within the Map expression. It runs recursively until it reaches a token matching a right bracket or the end of the file. The calls it can make while recursing is: Run the parseRoom function if the token matches the Room keyword token. Run the parseConnect function if the token matches the Connect keyword token. If the token matches none of the two keywords it returns an error, saying that it expected something of the Room or Connect type.

```

1 void parseMapBody(ASTNode* mapNode, Token* currentToken)
2 {
3     if (currentToken->token == T_RBRACE || currentToken->token == T_EOF)
4     {
5         return;
6     }
7
8     switch (currentToken->token)
9     {
10        case T_ROOM: parseRoom(mapNode, currentToken); break;
11        case T_CONNECT: parseConnect(mapNode, currentToken); break;
12        default: reportParserError("Connect | Room", currentToken);
13    }
14
15    parseMapBody(mapNode, currentToken);
16 }
17

```

Figure 5.22: The parseMapBody function

Parsing a Room expression: Like the mapExpr function, the parseRoom function uses the helper functions to check that the structure of tokens is correct, in this case for an Room element. It first checks that we have a correct Room keyword token. Afterwards it makes sure to store a copy of the room id, in the same way we did for the map id, since this will be needed later. Afterwards it checks if we have a correct identifier token followed by a correct semi-colon token. If all this is correct it uses the ASTCreateRoom function from the astree file (see section 5.3) to build the room node of the abstract syntax tree.

```

1 void parseRoom(ASTNode* mapNode, Token* currentToken)
2 {
3     int roomPos = currentToken->pos;
4     if (!consumeToken(currentToken, T_ROOM))
5         reportParserError("Room", currentToken);
6
7     const char* id = strdup(currentToken->value);
8
9     if (!consumeToken(currentToken, T_IDENTIFIER))
10        reportParserError("identifier", currentToken);
11
12    if (!consumeToken(currentToken, T_SEMICOLON))
13        reportParserError(";", currentToken);
14
15    ASTCreateRoom(mapNode, id, roomPos);
16 }
17

```

Figure 5.23: The parseRoom function

Parsing a Connection expression:

The parseConnect function works similarly to parseRoom and mapExpr to with some minor changes. First of all it does a double token check at the start to check both for a correct Connect keyword token, followed by a correct left bracket token. After this it stores a duplicate of the id like it was the case in parseRoom and mapExpr, and checks that we have a correct id token. Afterwards instead of using consumeToken we had to make a custom

token definition and scan it, since we allow the operator to be either a directional edge token or a bidirectional one. Then another id copy is made and this id is also checked to see if it is a correct token. Finally we do a double token check again, where we check that we have a correct right bracket token, followed by a correct semi-colon token. If everything is correct we use the `ASTCreateConnect` function from the `astree` file (see section 5.3) to build the connect node of the abstract syntax tree.

```

1 void parseConnect(ASTNode* mapNode, Token* currentToken)
2 {
3     int connectPos = currentToken->pos;
4     if (!consumeToken(currentToken, T_CONNECT) || !consumeToken(
currentToken, T_LPAREN))
5         reportParserError("(", currentToken);
6
7     const char* id = strdup(currentToken->value);
8
9     if (!consumeToken(currentToken, T_IDENTIFIER))
10        reportParserError("Identifier", currentToken);
11
12    TokenDef op = currentToken->token;
13
14    if (op != T_DIRECTED_EDGE && op != T_BIDIRECTIONAL_EDGE)
15        reportParserError("> | <->", currentToken);
16
17    scan(currentToken); // Consume edge token
18
19    const char* id2 = strdup(currentToken->value);
20
21    if (!consumeToken(currentToken, T_IDENTIFIER))
22        reportParserError("identifier", currentToken);
23
24    if (!consumeToken(currentToken, T_RPAREN) || !consumeToken(
currentToken, T_SEMICOLON))
25        reportParserError(") + ;", currentToken);
26
27    ASTCreateConnect(mapNode, id, op, id2, connectPos);
28 }
29

```

Figure 5.24: The `parseConnect` function

The final Parse: The final step of the whole parsing process is done in the "parse" function. Here we use the output from the `mapExpr` function to actually output the constructed abstract syntax tree. First an empty abstract syntax tree is constructed. This tree is then filled in with the output of `mapExpr`, where we make sure to set the output as the head of the tree. Finally the constructed tree is returned and the parsing process is done.

```

1 ASTree parse(Token* currentToken)
2 {
3     printf("Lexical analysis successful!\n");
4     ASTree tree = ASTInit();
5     ASTNode* mapNode = mapExpr(tree, currentToken);
6     tree->head = mapNode;
7     return tree;
8 }
9

```

Figure 5.25: The parse function

5.4.1 Parsing the Pseudo Code

After the pseudo code has gone through the lexer, the produced list of tokens seen on figure 5.15 is used in the parser to produce an abstract syntax tree. The tokens from the list are consumed in order to fill in the empty abstract syntax tree with nodes and child nodes matching the order of tokens. The visual representation of the tree produced from the pseudo code's tokens can be seen on figure 5.26.

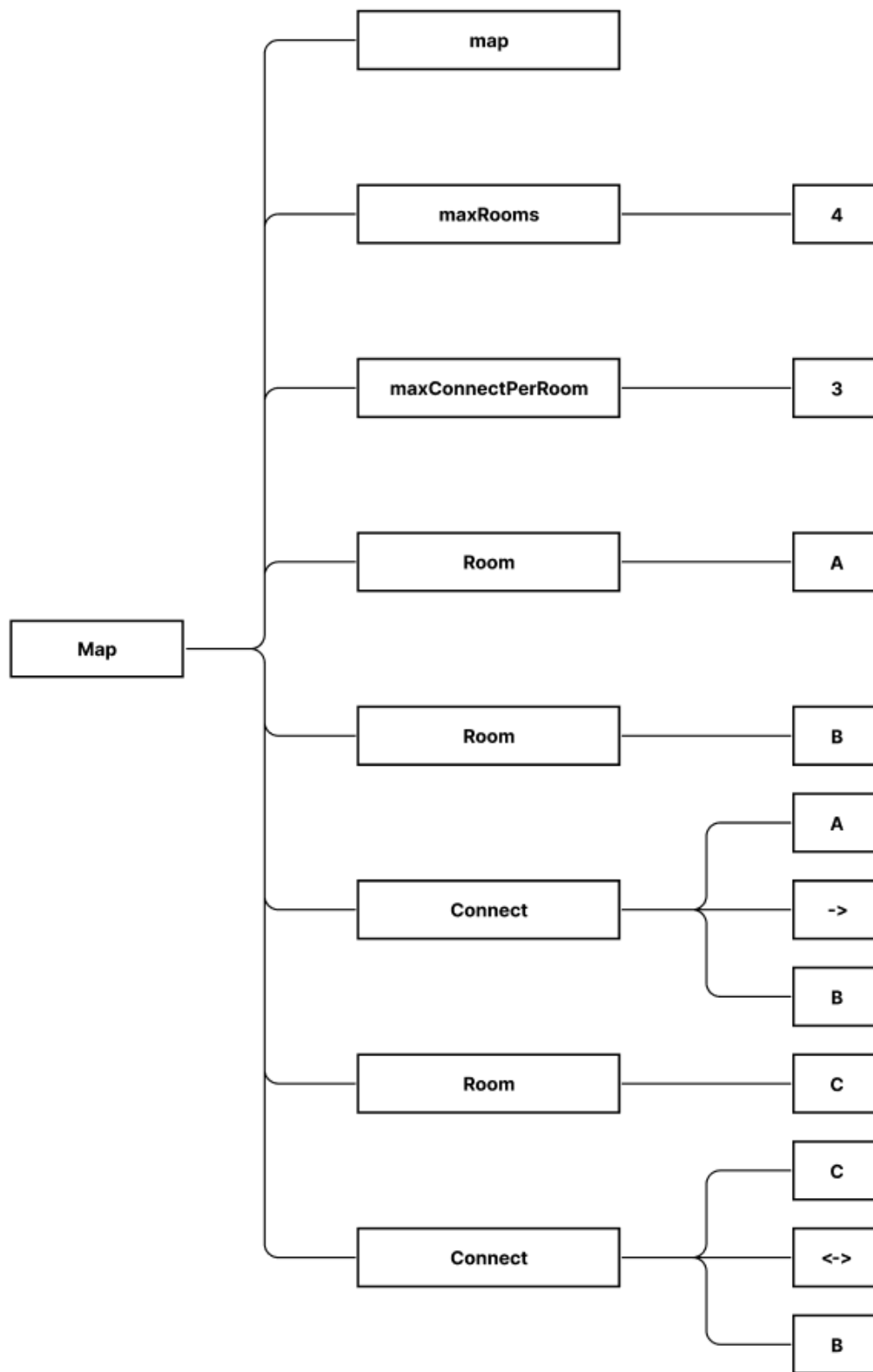


Figure 5.26: A visual representation of the abstract syntax tree produced for the pseudo code

5.5 Semantic Analysis

In the following sections, we will go over how the compiler does the semantic analysis, this will include type checking and constraint check.

5.5.1 Semantic Analysis Implementation

The semantic analysis is implemented as a separate component run on the abstract syntax tree (AST) from the parser component and is built around the concept of implementing a symbol table to keep track of the different variables. The semantic analyser will traverse through the abstract syntax tree, and for each node it will do a specific semantic check of the node depending on the type.

5.5.2 Symbol Table

The symbol table is implemented by defining two custom data structures: `SymbolTableEntry` and `SymbolTableStruct`. The `SymbolTableEntry` data structure, Represent a single entry in the symbol table and contains a pointer to the associated AST node. Through this AST location, it is possible to extract relevant information about the entry node such as id, type, and child nodes. The `SymbolTableStruct` data structure represents the entire symbol table, and contains an array of entries of the type `SymbolTableEntry`. It also keeps track of how many entries there is in the array (count) and the allocated memory for entries it can hold (capacity) these two variables are used to dynamically allocate memory for the entries array. Furthermore, the symbol table also holds the variables `room_constr` and `connect_constr` that are used to constraint the symbol table based on what is defined in the input file. For the symbol table there is also implemented a number of helper functions:

```
1 typedef struct SymbolTableEntry {
2     const ASTNode* ast_location;
3 } SymbolTableEntry;
4
5 typedef struct SymbolTableStruct {
6     SymbolTableEntry* entries;
7     size_t count;
8     size_t capacity;
9     int room_constr;
10    int connect_constr;
11 } *SymbolTable;
12
```

Figure 5.27: Symbol table structure

Adding to symbol table:

The AddSymbolTable function is used to add a new entry to the symbol table. The function takes two arguments, the symbol table in which the entry should be added, and a pointer to the AST node that should be added. The function starts by checking if the memory capacity of the symbol table is reached and if so it will double the capacity of the symbol table. After doing the memory check it will increment the symbol table count and store a new entry with the given AST node pointer.

```
1 void AddSymbolTable(SymbolTable table, const ASTNode* node)
2 {
3     if (table->count >= table->capacity)
4     {
5         size_t new_cap = table->capacity * 2;
6         SymbolTableEntry* new_entries = realloc(
7             table->entries,
8             new_cap * sizeof(SymbolTableEntry));
9         table->entries = new_entries;
10        table->capacity = new_cap;
11    }
12
13    table->entries[table->count++] = (SymbolTableEntry){
14        .ast_location = node,
15    };
16 }
17
```

Figure 5.28: AddSymbolTable code

Look up in the symbol table

These two functions are used to look up in the symbol table to see if and variable is already stored in the symbol table. LookUpSymbolTable takes an id and looks it up in a given symbol table by using a for loop to go through each entry in the symbol table and compare the id of the entry with the given id.

```
1 const SymbolTableEntry* LookUpSymbolTable(const SymbolTable table, const
    char* id)
2 {
3     if (!table || !id) return NULL;
4
5     for (size_t i = 0; i < table->count; i++)
6     {
7         if (strcmp(table->entries[i].ast_location->children[0]->data, id)
8             == 0) return &table->entries[i];
9     }
10    return NULL;
11 }
```

Figure 5.29: The function to look up the symbol table

LookUpConnectSymbolTable works in a similar manner, however, instead of only taking one id, it takes the id of two rooms it trying to connect and check if any connection of that combination has already been stored in the symbol table.

```

1 int LookUpConnectSymbolTable(const SymbolTable table, const char*
   first_room, const char* second_room)
2 {
3     if (!table || !first_room || !second_room) return 0;
4
5     for (size_t i = 0; i < table->count; i++)
6     {
7         if (table->entries[i].ast_location->type == AT_CONNECT){
8             if (table->entries[i].ast_location->children[1]->type ==
AT_BIDIRECTIONAL_EDGE)
9                 {
10                     if (strcmp(table->entries[i].ast_location->children[0]->
data, first_room) == 0
11                         && strcmp(table->entries[i].ast_location->children
[2]->data, second_room) == 0
12                         || strcmp(table->entries[i].ast_location->children
[2]->data, first_room) == 0
13                         && strcmp(table->entries[i].ast_location->children
[0]->data, second_room) == 0
14                         ) return 1;
15                 }
16             else if (strcmp(table->entries[i].ast_location->children[0]->
data, first_room) == 0
17                         && strcmp(table->entries[i].ast_location->children
[2]->data, second_room) == 0
18                         ) return 1;
19         }
20     }
21     return 0;
22 }
23

```

Figure 5.30: The function to look up the connection symbol table

Checking constraints:

The component uses various helper functions to handle the constraints that might be implemented in the source code. One of the functions will enable the constraint on the symboltable by declaring the constraint value to the SymbolTableStruct. The main logic for checking the if the constrains are followed are done in the function CheckConstrSymbolTable that takes the complete symbol table as argument:

Line 3: The function first check if the constraint has been enabled. Then uses helper function CountRoomsSymbolTable to count number of nodes saved in symboltable with type Room and check if its under declared constraint value.

Line 5-6: Here the function uses the helper function FindMaxConnectSymbolTable to find the room that has the highest amount connections to other rooms in the symbol table is connected to. Then the function checks if any constraint on connection has been declared and if so if they dont violate the constraint value.

line 8-11: Here the function defines the return value. this value is used later for the assembly code to tell the program how many connection that a room can have. The return value is defined as the nearest power of two number above eight and the maximum amount of connection defined to a room.

```

1 int CheckConstrSymbolTable (SymbolTable table)
2 {
3     if (table->room_constr != -1 && table->room_constr <
        CountRoomsSymbolTable(table)) return -1;
4
5     int max_connect = FindMaxConnectSymbolTable(table);
6     if (table->connect_constr != -1 && table->connect_constr <
        max_connect) return -1;
7
8     int i = 8;
9     while (i < max_connect)
10         i *= 2;
11     return i;
12 }
13

```

Figure 5.31: Constraint check logic

Initilize and freeing symbol table

Additional there is also defined two helperfunction: `InitSymbolTable` for initilization of the symbol table with a given capacity, where room count is set to 0 and both constr is set to -1, this is done so they are disenabled as default. `FreeSymbolTable` is for freeing the whole table and its entries from memory.

5.5.3 Semantic Analyzer

The implementation of the semantic analyser is build around the main function called `TypeCheck`, this function starts by initializing the symbol table at an initial capacity of 20, this is done so the program does not have to update the capacity multiple times at low capacity of symbol table. Then the function starts traversing through the AST from the head node, by using the Helper function `TraverseAST`, this function will recursively go through each node of the tree and check for the type of the node, and depending on that call associated semantic check function, if no sematic rules are violated it will also add the node to the symbol table using the `AddSymbolTable`. If encountering a node with a constraint type it will call the `ConstrSymbolTable` function to enable the constraint on the symbol table.

Semantic check of Map:

A helper function is used to do the check for nodes with the Map type. The function will use the `LookUpSymbolTable` function (see section 5.5.2) to see if any Map with same identifier is already stored in symbol table. since the program at its current state does not support multiple Map declaration in one file this check is redundant to the semantic analysis. However the identifier of the Map is still stored in the symbol table to ensure no Room is given identical identifier.

Semantic check of Room:

A helper function that works in identical way of the Map check function to see if any Room with identical identifier is stored in symbol table. If a identical identifier is stored in symbol table, the function checks if the identifier stored is of type Room to give more specific error messages to users.

```

1 void checkRoom(SymbolTable table, const ASTNode* node)
2 {
3     const SymbolTableEntry* val = LookUpSymbolTable(table, node->children
4     [0]->data);
5     if (val != NULL)
6         if (val->ast_location->type == AT_ROOM)
7         {
8             reportSemanticError(ERR_SEMANTIC, node->pos, "Room is already
9             declared");
10        }
11        else
12        {
13            reportSemanticError(ERR_SEMANTIC, node->pos, "Room name
14            already declared for another type");
15        }
16    }
17 }

```

Figure 5.32: checkRoom code implementation

Semantic check of Connect: A helper function is called when the TraverseAst encounter a Connect type.

Line 3-7: The function starts by doing a look up of both rooms, then it checks if both Rooms was stored in the symbol table if not give a custom error message to the user.

Line 9-13 Then it checks if a connection with identical combination is stored in symbol table, if it is a bi-directional connection it checks both combination, if already stored the program stops and return costum error message.

Line 15-16: Then it checks if the given Rooms are of type Room, e.g. if a connection between a Room and Map identifier is trying to be made.

Line 18-19: Finally it checks if the rooms are the same, since it should not be able to connect a room to itself.

```

1 void checkConnection(SymbolTable table, const ASTNode* node)
2 {
3     const SymbolTableEntry* first_room = LookUpSymbolTable(table, node->
children[0]->data);
4     const SymbolTableEntry* second_room = LookUpSymbolTable(table, node
->children[2]->data);
5
6     if (!first_room || !second_room)
7         reportSemanticError(ERR_SEMANTIC, node->pos, "Cant connect
undeclared Room reference");
8
9     if (LookUpConnectSymbolTable (table, node->children[0]->data, node->
children[2]->data))
10        reportSemanticError(ERR_SEMANTIC, node->pos, "Connection already
declared");
11
12    if (node->children[1]->type == AT_BIDIRECTIONAL_EDGE &&
LookUpConnectSymbolTable (table, node->children[2]->data, node->
children[0]->data))
13        reportSemanticError(ERR_SEMANTIC, node->pos, "Connection already
declared");
14
15    if (first_room->ast_location->type != AT_ROOM || second_room->
ast_location->type != AT_ROOM)
16        reportSemanticError(ERR_TYPE, node->pos, "Expected Room types");
17
18    if (strcmp(node->children[0]->data, node->children[2]->data) == 0)
19        reportSemanticError(ERR_SEMANTIC, node->pos, "Can't connect a
Room to itself");
20
21 }
22

```

Figure 5.33: The checkConnection code implementation

Error support and visualization of symbol table:

At all the semantic check if a rule is violated the reportSemanticError function is called, this function calls the reportError function (see section 5.1.3). Before freeing the symbol table the main function TypeCheck also calls PrintSymbolTable which for each entry print the type of the entry and the memory location.

5.6 Intermediate Language

After parsing and typechecking our DSL, we translate the AST into our Intermediate Language (IL), which is essentially a flat sequence of simple instructions. By abstracting away nesting, punctuation, and the details of the AST, the IL serves as a bridge between our high-level DSL and target assembly code. By transforming all our different AST node types (maps, rooms, connections) into a single, flat sequence of instructions, the machine code generation stage only needs to deal with one, simple data structure.

In our compiler, IL is essentially a dynamic array (InstructionTable) of instructions that carries an instruction code (e.g. IR_DECL_MAP, IR_DECL_ROOM, IR_DECL_CONNECT) and an array of string arguments. The whole structure can be seen in 5.34 below:

```

1 typedef enum {
2     IR_DECL_MAP,
3     IR_DECL_ROOM,
4     IR_DECL_CONNECT,
5 } InstructionCode;
6
7 typedef struct {
8     InstructionCode InstrCode;
9     char **args;
10    int argc;
11 } Instruction;
12
13 typedef struct {
14     Instruction *entries;
15     int count;
16     int capacity;
17 } *InstructionTable;
18

```

Figure 5.34: Intermediate language structure

AST tree-walk: The main function of our IL generation is a simple tree-walk over AST (5.35). We look at the type of AST node (AT_MAP, AT_ROOM, or AT_CONNECT) and call the appropriate helper. After handling the current node, we traverse each of its children in turn, so every AST node is visited exactly once.

```

1 void generate_il(ASTNode *node, InstructionTable table){
2     if(!node) return;
3
4     switch(node->type){
5         case AT_MAP:
6             handle_map_node(node, table);
7             break;
8
9         case AT_ROOM:
10            handle_room_node(node, table);
11            break;
12
13        case AT_CONNECT:
14            handle_connect_node(node, table);
15            break;
16    }
17
18    for(int i = 0; i < node->child_count; i++){
19        generate_il(node->children[i], table);
20    }
21 }
22

```

Figure 5.35: The generate_il function

Map Declaration Handler: If it is a Map node, it always has at least one child - the map's identifier. We use it to build the IL Instruction using `instr_create()` and then append the new instruction onto our `InstructionTable` using `instr_add()`.

Line 2: Safety check to see if map has at least one child.

Line 6-7: Map identifier is extracted from the AST node and put into an argument array.

Line 9-10: creates an instruction for the Map node and adds it to the instruction table

```
1 void handle_map_node(ASTNode *node, InstructionTable table) {
2     if (node->child_count < 1) return;
3
4     ASTNode *mapIdNode = node->children[0];
5
6     char *mapId = mapIdNode->data;
7     char *args[] = { mapId };
8
9     Instruction instr = instr_create(IR_DECL_MAP, args, 1);
10    instr_add(table, instr);
11 }
12
```

Figure 5.36: The handle_map_node function

Room Declaration Handler: For a Room node, we have exactly the same pattern as for maps - we pull out the rooms name, emit an IR_DECL_ROOM instruction, and append it.

```
1 void handle_room_node(ASTNode *node, InstructionTable table) {
2     if (node->child_count < 1) return;
3
4     ASTNode *roomIdNode = node->children[0];
5
6     char *roomId = node->children[0]->data;
7     char *args[] = { roomId };
8
9     Instruction instr = instr_create(IR_DECL_ROOM, args, 1);
10    instr_add(table, instr);
11 }
12
```

Figure 5.37: The handle_room_node function

Connections Handler: A Connect node has three children: sourceId, operator and destinationId. So first we emit the forward connection from src to dst, and if the operator is bidirectional (<->), we append a second instruction swapping src and dst.

Line 2: safety check to see if connection have three children.

Line 4-6: Extract the information from the child nodes

Line 8-11: Declare the source and destination ids and add to a argument list

Line 13-14: creates instruction for connection and add it to instruction table.

Line 16-20: check if bi directional operator and if so also add instruction for connection from destination to source.

```

1 void handle_connect_node(ASTNode *node, InstructionTable table) {
2     if (node->child_count < 3) return;
3
4     ASTNode *srcNode = node->children[0];
5     ASTNode *opNode  = node->children[1];
6     ASTNode *dstNode = node->children[2];
7
8     char *src = srcNode->data;
9     char *dst = dstNode->data;
10
11     char *args[] = { src, dst };
12
13     Instruction instr = instr_create(IR_DECL_CONNECT, args, 2);
14     instr_add(table, instr);
15
16     if (opNode->type == AT_BIDIRECTIONAL_EDGE) {
17         char *args[] = {dst, src };
18
19         Instruction instr = instr_create(IR_DECL_CONNECT, args, 2);
20         instr_add(table, instr);
21     }
22 }
23

```

Figure 5.38: The handle_connect_node function

If we look at our pseudo code example 4.1 and the resulting AST 5.26, our IL generation emits exactly one IL instruction per map/room/connection AST node (and an extra reverse connection when it sees <->). We create the following instructions:

- We declare the Map map.
- We declare Rooms A, B, then link A to B.
- We declare Room C and then link C to B and B to C as it is bidirectional.

As shown in Table 5.1 below, the IL generation maps each AST node to its respective instruction:

Index	Instruction Code	Arguments
0	IR_DECL_MAP	"map"
1	IR_DECL_ROOM	"A"
2	IR_DECL_ROOM	"B"
3	IR_CREATE_CONNECTION	"A", "B"
4	IR_DECL_ROOM	"C"
5	IR_CREATE_CONNECTION	"C", "B"
6	IR_CREATE_CONNECTION	"B", "C"

Table 5.1: Generated Intermediate Language (IL) from the AST

Note that Map constraints (maxRooms, maxConnectPerRoom) are parsed into the AST but currently not lowered into IL. Furthermore, the arrow operators (->, <->) are emitted as one or two IL instructions with two arguments for the source and destination rooms.

To add new features, such as room properties (e.g., *difficulty*, *lock*, etc.), we can extend the existing AST handlers or introduce new ones to capture these additional child nodes. Correspondingly, new `InstructionCode` entries are added to represent these properties in the IL table (5.34). So, our `InstructionTable` would simply grow with more entries, while its overall structure remains consistent - a flat array of instructions with their arguments.

5.7 Machine Code

After the intermediate language translation, we defined each instruction in `x86_64`, for targeting Linux, macOS, and Windows and handwritten assembly to validate through an depth first search algorithm and gamify the traversal process.

The translated instructions turned into two parts of assembly code: one for constructing the graph data, another for configuring runtime parameters for the handwritten. Additionally, we also generate a C interface enabling communication with assembly code in C.

We picked Netwide Assembler (NASM) [24] due to the cross-platform functionality. NASM can specify the target operating system and convert the assembly code into platform-specific formats, including `macho64` for macOS, `win32/win64` for Windows, and `elf64/elf32` for Linux.

5.7.1 Code Generation

The code generation is designed to transform the intermediate language into assembly code. First, it constructs the map data structure using the `generate_map` function. Next, it creates a configuration file for the assembly depth-first search via `generate_config`. Additionally, it generates a C interface through `generate_interfaces`.

5.7.1.1 Generate Map

The core component of our code generation is the function `GenerateMap`, which models the layout and connectivity between rooms. Its primary purpose is to generate a graph-like data structure in machine code, following the `x86_64` architecture. The map generation process is divided into two distinct phases: room discovery and exposure, followed by room instantiation and connection generation.

Phase 1: Room Discovery and Exposure: This phase extracts essential map metadata, such as the total room count, and exposes room labels for external linkage.

- **Line 1:** A basic check ensures that the instruction table contains at least one instruction before proceeding.
- **Lines 3–5:** The `countRooms` function traverses the instruction table. For every `IR_DECL_ROOM` instruction, it increments a counter. Once complete, it returns the total room count and writes NASM preprocessor definitions:
 - `%define ID_LEN` is defined using the maximum token data length.
 - `%define ROOM_COUNT` represents the total number of rooms discovered.
- **Lines 10–20:** The `getRoomNames` function performs a similar traversal but returns a list of room name strings. It first declares a global entry symbol, followed by each room name in a loop. This exposes the labels for external linking and access within the NASM code.

```
1      if (table->count >= 1)
2  {
3      int room_count = countRooms(table);
4      fprintf(file, "%define ID_LEN %d\n", ID_LEN);
5      fprintf(file, "%define ROOM_COUNT %d\n\n", room_count);
6
7      fprintf(file, "section .data\n");
8
9      char** roomNames = getRoomNames(table);
10
11     fprintf(file, "global entry");
12     for (int i = 0; i < roomCount; i++)
13     {
14         fprintf(file, ", room_%s", roomNames[i]);
15     }
16     fprintf(file, "\n\n");
17
18     free(roomNames);
19 }
20
```

Figure 5.39: Code Gen count and global

Phase 2: Room Instantiation and Connection Generation: This phase handles the creation of each room structure, resolves its connections, and writes the corresponding NASM assembly code.

- **Lines 1–13:** This block sets up the global entry label. A flag variable, `hasEntry`, ensures the label is assigned only once. The loop skips any non-room declarations until it encounters the first `IR_DECL_ROOM` instruction, which is then marked as the entry point.
- **Lines 15–27:** For each room declaration, a `Room` structure is dynamically allocated and initialized. This includes assigning the room ID and allocating memory for its potential connections. If memory allocation fails, the program reports the error and exits.

- **Line 29:** The `getConnections` function is called to populate the rooms connection list. It scans subsequent instructions for `IR_DECL_CONNECT` entries that reference the current rooms ID. Only connections referencing the current room are considered.
- **Lines 30–34:** The `writeRoomAssembly` and `writeConnectAssembly` functions output the NASM representation of the room and its connections. Each symbol is prefixed with `room_` to prevent naming conflicts with NASM keywords. Internally, these functions use loops and formatted output to write all relevant properties and metadata to the file, and then free the rooms memory afterward.

```

1  int hasEntry = 0;
2
3  for (int i = 0; i < table->count; i++)
4  {
5      if (table->entries[i].InstrCode != IR_DECL_ROOM)
6          continue;
7
8      if (!hasEntry)
9      {
10         fprintf(file, "align 8\n");
11         fprintf(file, "entry: dq room_%s\n", table->entries[i].args
[0]);
12         hasEntry = 1;
13     }
14
15     Room* currentRoom = calloc(1, sizeof(Room));
16     currentRoom->id = table->entries[i].args[0];
17     currentRoom->connCount = 0;
18
19     currentRoom->connections = calloc(max_connections_global, sizeof(
char*));
20
21     if (!currentRoom->connections)
22     {
23         perror("Memory allocation failed for connections\n");
24         free(currentRoom);
25         fclose(file);
26         return;
27     }
28
29     getConnections(table, currentRoom, i + 1);
30     writeRoomAssembly(currentRoom, file);
31     writeConnectAssembly(currentRoom, file);
32
33     free(currentRoom->connections);
34     free(currentRoom);
35 }
36

```

Figure 5.40: Code Gen map instantiation

Figure 5.41 shows the NASM output generated from the pseudo code in Figure 4.1. In Phase 1, two key constants are defined: `ID_LEN` and `ROOM_COUNT`. This example includes the entry point and three declared rooms. A fixed identifier length of 32 bytes is used to ensure a consistent and aligned memory layout.

In Phase 2, the data structures are aligned to match the requirements of the target CPU architecture. The entry label is both initialized and aligned to reference the first room. Each room definition is similarly CPU-aligned and includes the room's name, necessary padding for alignment, and a list of its connections.

```
1 %define ID_LEN 32
2 %define ROOM_COUNT 3
3
4 section .data
5 global entry, room_A, room_B, room_C
6
7 align 8
8 entry: dq room_A
9
10 align 8
11 room_A:
12     db "A"
13     times 31 db 0
14     dq room_B
15     times 7 dq 0
16
17 align 8
18 room_B:
19     db "B"
20     times 31 db 0
21     dq room_C
22     times 7 dq 0
23
24 align 8
25 room_C:
26     db "C"
27     times 31 db 0
28     dq room_B
29     times 7 dq 0
30
```

Figure 5.41: Map data

5.7.1.2 Generate Config

To maintain a clear separation of concerns in the assembly code, we chose to generate a dedicated `config.asm` file. This ensures that the main map output only focus on the graph structure, while `config.asm` handles the configuration necessary for the depth-first search (DFS) algorithm.

The configuration file initializes a `visited_count` variable, which the DFS uses to track how many rooms have been visited during traversal. Additional variables are also defined to support the DFS stack, utilizing the known `ROOM_COUNT` to statically allocate the required memory at compile time, thus avoiding any dynamic memory management.

Generating the config:

- **Lines 4–7:** Initializes the file path and opens the file for writing. This sets up where the configuration will be saved.
- **Lines 8–9:** Calculates the number of rooms using the `countRooms` function (covered in the map generation section) 5.7.1. This value is used to define the stack size, incremented by one to allocate for a terminal null connection required by the algorithm and external use.
- **Lines 11–20:** Writes the necessary data and uninitialized sections for the DFS (Depth-First Search) configuration. This includes the visited count, stack size, and pointers such as `stack_top` which refers to the top of the stack structure.

```
1 void generate_config(InstructionTable table)
2 {
3
4     char fileLoc[MAX_PATH_SIZE] = {0};
5     const char* path[]          = {ASM_DIR, "config.asm"};
6     safe_multi_strcat(fileLoc, path, 2, MAX_PATH_SIZE);
7     FILE* file                  = fopen(fileLoc, "w");
8     int room_count = countRooms(table);
9     int stack_size = room_count + 1; // Leave room for terminal
    connection 0
10
11     fprintf(file, "section .data\n\n");
12     fprintf(file, "    visited_count    dq 0\n\n");
13
14     fprintf(file, "section .bss\n\n");
15     fprintf(file, "    stack_size      equ %d\n", stack_size);
16     fprintf(file, "    stack          resq stack_size\n");
17     fprintf(file, "    stack_top      resq 1\n");
18     fprintf(file, "    visited_rooms  resq stack_size\n");
19
20     fclose(file);
21 }
22
```

Figure 5.42: Generate configuration file function

Figure 5.43 shows an example of the NASM output generated from the corresponding pseudocode 4.1. The `.data` section initializes a counter to track visited rooms during DFS traversal, while the `.bss` section allocates space for the stack and other structures based on the room count. Since there are 3 rooms, the stack size is set to 4 leaving room for a terminal connection required by the algorithm.

```

1 section .data
2
3     visited_count    dq 0
4
5 section .bss
6
7     stack_size       equ 4
8     stack             resq stack_size
9     stack_top         resq 1
10    visited_rooms     resq stack_size
11

```

Figure 5.43: Map data

5.7.1.3 Generate Interfaces

For the C demo included in the attachment, we created an algorithm that generates a C interface to demonstrate how the machine code can be integrated with an external language. This interface includes global preprocessor variables to expose graph metadata, reducing the need for manual setup. It also defines a struct called `Room`, which represents a node in the graph, and declares each room as `extern` so they can be linked with the `map.asm` file generated by the algorithm shown in Figure 5.41.

Generating the interface:

- **Lines 3–6:** Sets up the file path and opens the output header file, similar to the setup in `config.asm`.
- **Lines 10–17:** Writes preprocessor macros that define metadata such as the room count, maximum number of connections, and ID length. Then, it defines the `Room` struct, including the room name and its possible connections.
- **Lines 20–26:** Declares the entry room as an external pointer, followed by a loop that writes an `extern` declaration for each room in the graph. These room names are retrieved via the `getRoomNames` function used during map generation (see Figure 5.40). This ensures all rooms are accessible for linkage with `map.asm` assembly output.

```

1 void writeCInterface(char** rooms, int roomCount, int roomNameSize, int
  maxConnections)
2 {
3     char fileLoc[MAX_PATH_SIZE] = {0};
4     const char* path[]          = {GENERATED_DIR, "/interfaces/
mf_interface.h"};
5     safe_multi_strcat(fileLoc, path, 2, MAX_PATH_SIZE);
6     FILE* file = fopen(fileLoc, "w");
7
8     fprintf(file, "#pragma once\n\n");
9
10    fprintf(file, "#define ROOM_COUNT %d\n", roomCount);
11    fprintf(file, "#define MAX_CONNECTION %d\n", MAX_CONNECTIONS);
12    fprintf(file, "#define ID_LEN %d\n", ID_LEN);
13
14    fprintf(file, "typedef struct Room {\n");
15    fprintf(file, "    char name[ID_LEN];
16    fprintf(file, "    struct Room* connections[MAX_CONNECTION];
17    fprintf(file, "} Room;\n\n");
18
19    // Write extern declarations for all rooms (example)
20    fprintf(file, "extern Room* entry;\n");
21
22    for (int i = 0; i < roomCount; i++)
23    {
24        fprintf(file, "extern Room room_%s;\n", rooms[i]);
25    }
26    fclose(file);
27 }
28

```

Figure 5.44: Function that writes a C header with room data and extern room declarations.

Figure 5.45 shows an example of the C interface output generated from the pseudo code in Figure 4.1. The file begins with preprocessor `#define` statements that represent room metadata, followed by the `Room` struct definition. After that, the entry pointer is declared, along with external declarations for the three rooms defined in the pseudocode.

```

1 #pragma once
2
3 #define ROOM_COUNT 3
4 #define MAX_CONNECTION 8
5 #define ID_LEN 32
6
7 typedef struct Room {
8     char name[ID_LEN];           // Room name
9     struct Room* connections[MAX_CONNECTION]; // Connections to other
rooms
10 } Room;
11
12 extern Room* entry;
13 extern Room room_A;
14 extern Room room_B;
15 extern Room room_C;
16

```

Figure 5.45: Header defining the `Room` struct and extern declarations for rooms.

5.7.2 Handwritten

The handwritten assembly code works on both Linux and macOS, and can also run on Windows using WSL (Windows Subsystem for Linux). All examples follow the Linux system conventions.

As mentioned earlier, the assembly is used to test the graph traversal and make it interactive. There are two main algorithms: DFS, which checks for dead ends, and `graph_walk`, which lets the user explore the graph step by step. The assembly code is organized into six main parts:

- **Core.asm** - Basic utility functions such as `print`, `exit_program` and input request.
- **Convert.asm** - Conversion functions to convert between int and strings.
- **Stack.asm** - Functions for a custom stack data structure, that utilizes the autogenerated `Config.asm` to determine the stack configurations.
- **DFS.asm** - A DFS algorithm utilizing `Config.asm` to store and track visited rooms during traversal and `Stacks.asm` for storing room queues.
- **Graph_walk.asm** - A traversal-based game that reacts to user input in the terminal.
- **Main.asm** - The entry point, gathering all five handwritten components along with the two assembly files generated from the C code.

To ensure the assembly code remains consistent, we follow the Linux x86_64 System V calling convention, where:

- The return value is stored in RAX
- The first six arguments are passed via registers in the following order: RDI, RSI, RDX, RCX, R8, and R9

Register	Usage	callee saved
%rax	temporary register; with variable arguments passes information about the number of vector registers used; 1 st return register	No
%rbx	callee-saved register	Yes
%rcx	used to pass 4 th integer argument to functions	No
%rdx	used to pass 3 rd argument to functions; 2 nd return register	No
%rsp	stack pointer	Yes
%rbp	callee-saved register; optionally used as frame pointer	Yes
%rsi	used to pass 2 nd argument to functions	No
%rdi	used to pass 1 st argument to functions	No
%r8	used to pass 5 th argument to functions	No
%r9	used to pass 6 th argument to functions	No
%r10	temporary register, used for passing a function's static chain pointer	No

Figure 5.46: Calling conventions

5.7.2.1 Core

The Core component includes utility functions such as `exit_program`, all print-related functions, and `scan` for capturing user input

Print

The `print` function is a simple utility used to visualize data in the terminal. Below is a breakdown of each step in NASM for figure 5.47:

- **Lines 2–4:** Save `rdi`, `rsi`, and `rdx` to preserve register values.
- **Line 6:** Copy the string pointer from `rdi` to `rsi`.
- **Line 7:** Calls `slen` to calculate the string length by storing the start address in one register and incrementing a second register until the null terminator is reached. The length is then calculated as the difference between the two and returned in `rax`.
- **Lines 9–11:** Prepare the `sys_write` system call:
 - `rdx` = length, `rdi` = `stdout`, `rax` = syscall number.
- **Line 12:** Execute the syscall to print the string to terminal.
- **Lines 14–16:** Restore the original values of the saved registers.
- **Line 18:** Return from the function.

```
1 print:
2     push rdi
3     push rsi
4     push rdx
5
6     mov rsi, rdi
7     call slen
8
9     mov rdx, rax
10    mov rdi, 1
11    mov rax, 1
12    syscall
13
14    pop rdx
15    pop rsi
16    pop rdi
17
18    ret
19
```

Figure 5.47: Print function

Other print variants exist to insert newlines, spaces, or print integers by converting them to strings and calling `print`.

Scan:

The scan function utilizes the Linux kernel's `sys_read` system call to retrieve user input from the terminal. Below is a breakdown of each step in NASM for figure 5.48.

- **Lines 2-4:** Save the values of the `rdx`, `rsi`, and `rdi` registers by pushing them onto the stack. This preserves their original values before the system call setup.
- **Lines 6-9:** The registers are initialized to set up a `sys_read` system call:
 - `rax` is set to 0, which corresponds to the read syscall number in the Linux kernel.
 - `rdi` is set to 0, indicating standard input (`stdin`) as the input source.
 - `rsi` is assigned the address of `input_buffer`, which specifies where the input will be stored.
 - `rdx` is set to `input_len`, which defines the maximum number of bytes to read.

The variables `input_buffer` and `input_len` are defined earlier in the program and provide the memory location and input length constraint.

- **Line 10:** Trigger the kernel to perform the read function
- **Line 12:** The last character in the input is replaced with 0 as the terminal value in the string
- **Line 13:** Put the input into the `RAX` register following the calling convention
- **Line 15-17:** Restore registers to original values by popping from the stack to the registers
- **Line 18:** Trigger return to finish the function

```
1 scan:
2     push    rdx
3     push    rsi
4     push    rdi
5
6     xor     rax, rax
7     xor     rdi, rdi
8     mov     rsi, input_buffer
9     mov     rdx, input_len
10    syscall
11
12    mov     byte [rsi + rax - 1], 0
13    mov     rax, rsi
14
15    pop     rdi
16    pop     rsi
17    pop     rdx
18    ret
19
```

Figure 5.48: Scan function

exit_program:

The `exit_program` function calls `sys_exit` to terminate the program. Below is a breakdown of each step in NASM for figure 5.49.

- **Line 2:** Move 60 into `rax`, corresponding to `sys_exit` in the Linux syscall table.
- **Line 3:** Move 0 into `rdi`, indicating a successful exit status.
- **Line 4:** Invoke the syscall to exit the program.

```
1 exit_program:
2     mov     rax, 60
3     mov     rdi, 0
4     syscall
5
```

Figure 5.49: Exit_program function

5.7.2.2 Convert

The `Convert` component includes functions for converting integers to strings and vice versa. Below is a breakdown of each step in NASM for figure 5.50.

int2str

- **Lines 2–5:** Save registers that will be used in the function.
- **Lines 7–10:** Set up registers for conversion the number, buffer end, null terminator, and base 10.
- **Lines 12–16:** Special case handling for the input 0.
- **Lines 18–25:** Loop through the number, extracting each digit and building the string backwards.
- **Line 28:** Set `rax` to point to the beginning of the resulting string.
- **Lines 29–32:** Restore saved registers.
- **Line 33:** Return from the function.

```

1 int2str:
2     push    rbx
3     push    rcx
4     push    rdx
5     push    rdi
6
7     mov     rax, rdi
8     mov     rbx, int_buffer + 1000
9     mov     byte [rbx], 0
10    mov     rcx, 10
11
12    test    rax, rax
13    jnz     .convert_loop
14    dec     rbx
15    mov     byte [rbx], '0'
16    jmp     .done
17
18 .convert_loop:
19     xor     rdx, rdx
20     div     rcx
21     dec     rbx
22     add     dl, '0'
23     mov     [rbx], dl
24     test    rax, rax
25     jnz     .convert_loop
26
27 .done:
28     mov     rax, rbx
29     pop     rdi
30     pop     rdx
31     pop     rcx
32     pop     rbx
33     ret
34

```

Figure 5.50: int2str function

str2int

str2uint works similarly to int2str, but in reverse. Instead of converting an integer to a string, it takes a numeric string and builds the integer value by parsing each character. It loops through the string, skipping whitespace, validating digits, and building the number using base 10 logic. If the input contains invalid characters, it returns -1 in rax to signal an error.

5.7.2.3 Stack

The **Stack** component contains a custom stack structure, including initialization, push, and pop operations. Below is a breakdown of each step in NASM for figure 5.51.

Function: `init_stack`

- **Lines 4–6:** Set up the loop to clear the stack memory. `RCX` is set to the stack size, and `RDI` points to the beginning of the stack.
- **Lines 9–11:** Loop through the memory block, zeroing each 8-byte slot.
- **Line 12:** Update `stack_top` to point to the start of the stack.

Function: `stack_push`

- **Lines 16–17:** Save registers and load the value to push.
- **Lines 19–23:** Load current stack top and check for overflow.
- **Lines 23–27:** Store the value on the stack, increment pointer.
- **Lines 29–31:** Restore saved registers and return.
- **Line 33:** If overflow occurs, call `exit_program`.

Function: `stack_pop`

- **Lines 36–38:** Load stack top and check if the stack is empty.
- **Lines 40–43:** Move pointer down, retrieve the value into `rax`, zero the popped memory, and update `stack_top`.
- **Lines 46–47:** If empty, return 0 in `rax`.

```

1 section .text
2
3 init_stack:
4     mov rcx, stack_size
5     mov rdi, stack
6     xor rax, rax
7
8 .clear_loop:
9     mov [rdi], rax
10    add rdi, 8
11    loop .clear_loop
12    mov qword [stack_top], stack
13    ret
14
15 stack_push:
16    push rdi
17    push rsi
18
19    mov rsi, rdi
20    mov rdi, [stack_top]
21
22    cmp rdi, stack + stack_size * 8
23    jae .overflow
24
25    mov [rdi], rsi
26    add rdi, 8
27    mov [stack_top], rdi
28
29    pop rsi
30    pop rdi
31    ret
32 .overflow:
33    call    exit_program
34
35 stack_pop:
36    mov rdi, [stack_top]
37    cmp rdi, stack
38    jbe .empty
39
40    sub rdi, 8
41    mov rax, [rdi]
42    mov qword [rdi], 0
43    mov [stack_top], rdi
44    ret
45 .empty:
46    xor rax, rax
47    ret
48

```

Figure 5.51: Custom stack implementation in assembly

5.7.2.4 DFS

The DFS component traverses the map using a depth-first search algorithm. Due to the size of the full implementation, only the core logic is discussed here.

The traversal starts by pushing the entry room onto the stack:

```
1 mov     rdi, [entry]
2 call    stack_push
```

The algorithm loops as long as the stack is not empty. On each iteration, the top room is popped and checked against the list of visited rooms:

```
1 .dfs_loop:
2     ; Check for completion
3     mov     rdi, [stack_top]
4     cmp     rdi, stack
5     je      .dfs_done
6
7     call    stack_pop
8     mov     rdi, rax
9     call    check_visited
10    cmp     rax, 1
11    je      .dfs_loop
```

If the room hasnt been visited, it is marked and printed, and its connections are added to the stack:

```
1     call    mark_room
2     push    rdi
3     mov     rdi, traverse_msg
4     call    print
5     pop     rdi
6     call    printf
7
8     call    get_connections
9     jmp     .dfs_loop
10
11 .dfs_done:
12     ret
```

Rooms are marked as visited by storing their pointers in an array and incrementing a counter that tracks the total number of visited rooms. The `get_connections` routine then scans each rooms connections and pushes only the unvisited ones onto the stack.

At the end of the traversal, the algorithm verifies whether all rooms were visited by comparing the visited count against the total `room_count`.

```
1 call     all_room_traversed
```

This alert if there are unreachable areas in the map. If some rooms werent visited, a warning message is printed.

5.7.2.5 Graph_Walk

This code implements a simple graph traversal where the user can explore connected rooms interactively. The program prompts the user to select a room to visit, and allows traversal through the graph based on room connections.

Data Section

The data section contains messages for prompts and error handling. These include room selection headers, input prompts, and error messages such as when no more rooms are available, or when an invalid index is entered.

```
1 section .data
2     select_msg:      db "", 10
3                     db "          SELECT AVAILABLE ROOMS          ", 10
4                     db "", 0
5     no_rooms:        db "No more rooms to explore from here.", 0
6     bad_input:       db "Invalid input - please try again", 0
7     input_prompt:    db "Enter room index or 'q' to quit: ", 0
8     quit_msg:        db "Traversal session ended", 0
9     oob_msg:         db "Index out of range - please enter a valid room
                        index", 0
```

Graph Walk

The graph_walk function starts from an entry room and loops through available connections and prompts user for input through print_room_options and select_room. If the room has no connections, it notifies the user and exits.

```
1 graph_walk:
2     mov     rsi, [entry]
3     room to entry
4
5     .traversal_loop:
6     mov     rdi, [rsi + ID_LEN]
7     test    rdi, rdi
8     jz      .no_connections
9
10    call     print_room_options
11    call     select_room
12    cmp     rax, -1
13    je      .exit
14    mov     rsi, rax
15    jmp     .traversal_loop
16 .no_connections:
17    mov     rdi, no_rooms
18    call     printf
19 .exit:
20    ret
```


Print Room Options

The `print_room_options` function displays available rooms (connections) with indices. It loops through the room connections and prints the names along with their indices.

```
1 print_room_options:
2     mov     rdi, select_msg
3     call    printf
4     mov     rdx, 0
5
6 .print_loop:
7     mov     rdi, [rsi + ID_LEN + rdx * 8]
8     test    rdi, rdi
9     jz      .done_printing
10    call    print
11    mov     rdi, rdx
12    call    print_num
13    call    print_line
14    inc     rdx
15    jmp     .print_loop
16 .done_printing:
17     ret
```

Select Room: The `select_room` function counts the room connections and then prompts the user to input a room index. If the input is valid (an integer within the valid range), it moves to the selected room. If the input is invalid, it handles errors in two ways: if the user enters a non-integer (character), it will notify them of the conversion failure, and if the index is out of range, it will prompt the user to enter a valid room index.

```
1 select_room:
2     ; Count connections
3     mov     rbx, 0
4 .count_connections:
5     mov     rdi, [rsi + ID_LEN + rbx * 8]
6     test    rdi, rdi
7     jz      .input_loop
8     inc     rbx
9     jmp     .count_connections
10 .input_loop:
11     mov     rdi, input_prompt
12     call    print
13     call    scan
14     mov     rdi, rax
15     cmp     byte [rdi], 'q'
16     je      .quit
17     call    str2uint
18     cmp     rax, rbx
19     jge     .out_of_bounds
20     mov     rax, [rsi + ID_LEN + rax * 8]
21     ret
22 .out_of_bounds:
23     mov     rdi, oob_msg
24     call    printf
25     jmp     .input_loop
26 .quit:
27     mov     rdi, quit_msg
28     call    printf
29     mov     rax, -1
30     ret
```

Exit Conditions

When the user selects 'q' or select_room has no rooms, the program exits, printing a message and signaling the end of the traversal session.

Chapter 6

Discussion

In this chapter, we will discuss different aspects of our project. The chapter will start with a discussion about how the language appeals to our target group and whether our problem statement has been solved. Afterwards, we will discuss the development of our language and the choices we made.

6.1 Appealing to Target Group

Our compiler is designed for game developers who need a simple procedural map generation without the complexity of full game engines. Focus on just three instructions: Map, Room, and Connect. The reason for these three commands was that we wanted to start out with a simple map like in a grid-based game, where there is one full map and different tiles (Rooms) connected to each other.

The compiler handles only the layout of maps, avoiding features such as graphics or object placement. This is intentional, as most developers use game engines for rendering and gameplay logic. Instead, our output provides a graph structure with rooms as nodes that can be imported into game engines such as Unity, Godot, and Unreal. Although we have not tested this with users yet, the design reflects common needs in map design, with the ability to create different nodes and connect them.

The starting goal of our project was to create a lightweight tool for map creation, but as the project progressed. Although optimization is often associated with large-scale development teams, due to better allocation of time resources, we saw its value for indie developers, too. Indie games often aim for accessibility, and an optimized map structure can help ensure smoother performance on lower-end hardware. Furthermore, many indie developers work with limited resources, so having efficient output from the start reduces the amount of time and effort needed later for performance tuning. [17]

6.2 Solving the Problem Statement

Our problem statement was: How can we design a programming language specifically tailored to generating maps for small-scale games?

To achieve this, we developed a simple domain-specific language (DSL) compiler with just three core commands: Map, Room, and Connect. This minimal design keeps things lightweight and easy to use, ideal for small-scale game projects that need quick setup and flexibility. We do, however, acknowledge that the simple nature of the language at the current state limits the possibilities for the users.

Since small-scale games often rely on preexisting engines like Unity, Godot, or Unreal, we proved our generated output is compatible with external environments. We did this by generating machine code that's importable into other languages. To demonstrate this, we created a C interface as well as C and Python demos to show how our system can plug into multiple languages, and is not locked into just one. This highlights the versatility of our language and how the generated map/graph structures can be integrated into various frameworks.

6.3 Features

Throughout the implementation, we ran into various challenges at each stage, from the front-end to the back-end of the compiler. The front-end, where the game developer interacts, is where we realized the importance of an error-handling system. Users can make typos, bad sentences not following the parsing rules, and bad type usage, so having a way to catch and report those mistakes was essential for the compiler. We built a feature that reports the location of errors directly in the terminal, making debugging easier.

However, our error handler design had a limitation: it calculated error positions using a total character offset, making the exact character tricky to pinpoint. While it gave the correct line number, the character position was not always spot on. Still, it got us close enough to help users track down their mistakes. Another feature we added to the front-end was support for constraint parameters. During implementation, we had the choice to handle the calculations in the front-end or later in the back-end at the intermediate language stage. We went with the front-end because it already knows where the constraint is defined, which made it easier to provide feedback location to the game developer.

Although the constraints did not add much to the current version of the compiler, they laid the groundwork for future features like auto or semi-auto map generation, explained in future works 7.6.

6.4 Libraries

At the start of the project, we experimented with different libraries to implement the lexer and parser, including Re2c[23], Flex[22], and Bison[10]. Both Re2c and Flex are lexer generators, but since they produced generated code, we had to write additional interfaces and workarounds to integrate them. Bison, a parser generator, relied on Flex, and the tight coupling between the two made development cumbersome. Given these challenges, along with our language’s minimalistic design (which only required generating a graph data structure without complex features like scoping or in-source function definitions), we opted for a handwritten compiler. Additionally, we were interested in learning the process of building a compiler from scratch, making the handwritten approach both practical and educational.

6.5 Test Integration

To facilitate the process of having multiple people working on the code base, we decided to implement both unit tests and integration tests for the front-end phases of the compiler. These tests helped to ensure that the components lived up to our requirements and improved the reliability and maintenance of the code. However, the tests were implemented relative late in the code development, after most features were already implemented, making the advantages of having these test implemented not be as effective to the overall code development, and more of a feature for future work on the code, where it can simplify the integration of additional features to the language.

Chapter 7

Future Work

In this chapter, we will go over the different aspects that can be developed further. We will cover different kinds of tests that could be applied to the language. Furthermore, we will go into features that could be added to the language.

7.1 Acceptance Testing

As mentioned in the discussion, the actual appeal of a language like this for game development is unknown. To verify its purpose, one method could be to conduct acceptance testing on the language, where users could be included in the development process. Because of the state of the language as it is, performing a user test might not make the most sense. Therefore, the process of verifying the appeal of the language would await the development of more features for the language.

7.2 Stress Testing

As language evolves and we test the functional correctness of our system through acceptance testing, we should also consider non-functional requirements and test our system's performance under heavy usage. It provides insights on how the current system performs under extreme conditions to better identify limitations of our system and optimize it. This type of testing reveals limitations such as memory management, runtime efficiency, and others. Conducting a stress test on our system could ensure that our system meets the requirements for stability, performance, and scalability.

7.3 Additional Operators

To improve the expressiveness of our language, in later iterations, we should consider more traditional programming operators such as equality and logical operators (`==`, `!=`, `&&`, `||`, `AND`, `OR`) and relational operators (`<`, `>`, `<=`, `>=`). Adding these operators would bring our language's syntax and semantics even closer to other common programming languages.

7.4 Interpreter

Our compiler translates source code into an intermediate language, which is then converted into an `x86_64` assembly executable. Machine code generates map structures from the source code. We can reuse our existing front-end and intermediate language in the future to build an interpreter on top of our architecture that would read intermediate language to generate map structures and translate language into HTML for live map visualization in the web browser. Adding an interpreter would allow a more user-friendly development process, utilizing visualization when creating maps, and it would provide faster feedback, especially when designing maps without needing full compilation.

7.5 Custom Room Types

The current language design supports a single `Room` type and has no support for handling properties. We could expand our language to support custom room types and allow developers to create their own room types with associated properties. Custom room types would allow developers to represent more immersive game scenarios. A `BossRoom` could contain properties such as `difficulty`, `bossType`, or a `TreasureRoom` could contain `lootContents`. This would make language more flexible and provide developers with the possibility to represent specific behaviours.

7.6 Semi-Auto and Auto Map Generation

Since we have support for parameters such as `maxRooms` and `maxConnectPerRoom`, we could extend our project with auto and semi-auto map generation, where we could take advantage of these parameters. Auto map generation would allow users to specify high-level parameters describing specifications for the map we are generating, our language would then generate an entire map following those specifications without needing to add rooms manually. On the other hand, semi-auto mode would generate rooms in a more collaborative way, allowing users to start by manually adding rooms and connecting them, the program would then expand upon it by generating additional rooms and connections following specifications. This hybrid way of generating maps provides control of what should be defined manually and generated automatically.

7.7 Asynchronous Map Generation

At the current state of our compiler, it processes only one source code file at a time and it is sufficient for small scale use cases, but if game requires multiple maps to be generated, it could take longer than expected and we can significantly reduce time required to generate multiple maps by adding support for asynchronous compilation. This would allow for loading maps required for the game efficiently. By adding multithreading, the compiler would accept multiple input source files and execute the compiler's pipeline to generate each map concurrently. This could be particularly useful in dynamically generated environments and could allow new areas/dimensions to be created and loaded in the background without interrupting the game.

Chapter 8

Conclusion

Our goal was to answer the problem statement:

"How can we design a programming language specifically tailored to generating maps for small-scale games?".

By making our language consist of three instructions: Map, Room, Connection, and offloading visual and gameplay mechanics to existing game engines, we produced a language and a compiler that is lightweight and could be ported to existing engines. Our compiler successfully:

- Provides a grammar that mimics the basic understanding of a map layout in terms of the arrangement of rooms connected by pathways, such as doors, tunnels, hallways, etc.
- Provides error messages to improve user experience, by giving messages on problems, and what needs fixing.
- Parses and performs semantic analysis on map descriptions, checking for undeclared or duplicate rooms and connections, and constraint violations during compile time.
- Produces machine code for multiple kernels and produces portable binaries with a C interface, as was tested via C and Python demo projects, to further support integration with existing game development tools.

Summing all of this, we have demonstrated that it is possible to make even a minimal DSL that could meet the needs of our target group of newcomers and indie developers, to help with game map creation and prototyping. Of course, at this moment, there are many limitations - lacking room attributes, visual feedback, and more that need to be developed further to improve its expressiveness. In our discussion and future works, we have outlined some of the next steps to improve the language, such as future features like adding an interpreter for live map previews, auto-generated maps, and asynchronous map generation. With these new features added, the system could eventually be evaluated by performing acceptance and stress testing.

Bibliography

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison Wesley, Aug. 2006, ISBN: 0321486811. [Online]. Available: <http://www.amazon.ca/exec/obidos/redirect?tag=citeulike09-20%5C&path=ASIN/0321486811>.
- [2] S. Bhatia, S. Kohli, S. A. Seshia, and A. Cheung, “Building code transpilers for domain-specific languages using program synthesis”, in *37th European Conference on Object-Oriented Programming (ECOOP 2023)*, K. Ali and G. Salvaneschi, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 263, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 38:1–38:30, ISBN: 978-3-95977-281-5. DOI: 10.4230/LIPIcs.ECOOP.2023.38. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2023.38>.
- [3] E. Blem, J. Menon, and K. Sankaralingam, “Power struggles: Revisiting the risc vs. cisc debate on contemporary arm and x86 architectures”, in *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*, 2013, pp. 1–12. DOI: 10.1109/HPCA.2013.6522302.
- [4] BorisTheBrave.com. “Dungeon generation in binding of isaac”. Accessed: 2025-22-05. (2020), [Online]. Available: <https://www.boristhebrave.com/2020/09/12/dungeon-generation-in-binding-of-isaac/>.
- [5] BorisTheBrave.com. “Dungeon generation in enter the gungeon”. Accessed: 2025-22-05. (2019), [Online]. Available: <https://www.boristhebrave.com/2019/07/28/dungeon-generation-in-enter-the-gungeon/>.
- [6] A. Deursen and P. Klint, “Domain-specific language design requires feature descriptions”, *Journal of Computing and Information Technology*, vol. 10, no. 1, pp. 1–17, Jan. 2002. DOI: 10.2498/cit.2002.01.01.
- [7] Epic Games. “Unreal engine - the world’s most open and advanced 3d creation tool”. Accessed: 2025-04-02. (2025), [Online]. Available: <https://www.unrealengine.com/>.
- [8] M. Garcia, F. Ortin, and J. Quiroga, “Design and implementation of an efficient hybrid dynamic and static typing language”, *Software: Practice and Experience*, vol. 46, no. 2, pp. 199–226, 2016. DOI: <https://doi.org/10.1002/spe.2291>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.2291>.
- [9] GCC Team. “Gcc weppage”. (2025), [Online]. Available: <https://gcc.gnu.org/>.
- [10] GNU. “Gnu bison”. (2025), [Online]. Available: <https://www.gnu.org/software/bison/>.

- [11] E. Goh, O. Al-Tabbaa, and Z. Khan, "Unravelling the complexity of the video game industry: An integrative framework and future research directions", *Telematics and Informatics Reports*, vol. 12, p. 100 100, 2023, issn: 2772-5030. doi: <https://doi.org/10.1016/j.teler.2023.100100>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2772503023000609>.
- [12] ispmanager.com. "The history of game engines from assembly coding to photorealism and ai". Accessed: 2025-22-05. (2024), [Online]. Available: <https://dev.to/ispmanager/the-history-of-game-engines-from-assembly-coding-to-photorealism-and-ai-d9h>.
- [13] T. Kosar, N. Oliveira, M. Mernik, *et al.*, "Comparing general-purpose and domain-specific languages: An empirical study", *Computer Science and Information Systems*, vol. 7, no. 2, pp. 247–264, May 2010. doi: 10.2298/CSIS1002247K.
- [14] A. Latif, F. Azam, M. W. Anwar, and A. Zafar, "Comparison of leading language parsers antlr, javacc, sablecc, tree-sitter, yacc, bison", in *2023 13th International Conference on Software Technology and Engineering (ICSTE)*, 2023, pp. 7–13. doi: 10.1109/ICSTE61649.2023.00009.
- [15] LLVM Developer Group. "Clang weppage". (2025), [Online]. Available: <https://clang.llvm.org/>.
- [16] MediaCat. "Dentsu: gaming is bigger than music and movies combined". Accessed: 2025-22-05. (2024), [Online]. Available: <https://mediacat.uk/dentsu-gaming-is-bigger-than-music-and-movies-combined/>.
- [17] Mishal, "Game optimization challenges and solutions for indie developers", 2024. [Online]. Available: <https://lyn-shailer.co.uk/game-optimization-challenges-and-solutions-for-indie-developers/>.
- [18] T. Æ. Mogensen, *Introduction to compiler design, third edition*, 2024.
- [19] T. Æ. Mogensen, *Programming language design and implementation*, 2022.
- [20] F. Ortin and M. Garcia, "A programming language that combines the benefits of static and dynamic typing", in *Software and Data Technologies. ICSoft 2010*, ser. Communications in Computer and Information Science, J. Cordeiro, M. Virvou, and B. Shishkov, Eds., vol. 170, Berlin, Heidelberg: Springer, 2013, pp. 59–73. [Online]. Available: https://doi.org/10.1007/978-3-642-29578-2_5.
- [21] B. Roziere, M.-A. Lachaux, L. Chausson, and G. Lample, "Unsupervised translation of programming languages", ser. NIPS '20, Vancouver, BC, Canada: Curran Associates Inc., 2020, pp. 20 601–20 611, ISBN: 9781713829546.
- [22] F. development team. "Flex github page". (2024), [Online]. Available: <https://github.com/westes/flex>.
- [23] re2c development team. "Re2c webpage". (2025), [Online]. Available: <https://re2c.org/>.
- [24] T. N. development team. "Nasm webpage". (2025), [Online]. Available: <https://www.nasm.us/>.
- [25] Unity Technologies. "Unity - real-time development platform". Accessed: 2025-04-02. (2025), [Online]. Available: <https://unity.com/>.
- [26] Valve Corporation, *Steam*, 2025. [Online]. Available: <https://store.steampowered.com/>.

- [27] VG Insights, “Global indie games market report 2024”, VG Insights, Tech. Rep., Oct. 2024. [Online]. Available: <https://vginsights.com/insights/article/global-indie-games-market-report-2024>.
- [28] B. Wang, A. Kolluri, I. Nikoli, T. Baluta, and P. Saxena, “User-customizable transpilation of scripting languages”, *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, pp. 201–229, Apr. 2023. DOI: [10.1145/3586034](https://doi.org/10.1145/3586034). [Online]. Available: <https://doi.org/10.1145/3586034>.